

浙江大学

ZHEJIANG UNIVERSITY



项目设计报告

动态仓储环境下多 AMR 任务分配与路径协同调度优化模型研究

课程名称 运筹学

小组成员 李彦霖（组长），朱熠正，江亚楠

倪振昊，张铭浥

学 院 控制科学与工程学院

指导教师 赵斐

完成日期 2026 年 6 月 20 日

目录

摘要	3
1 问题背景与实现目标	3
1.1 现实背景	3
1.2 研究目标	3
2 系统数据与模块接口	4
2.1 二维仓库建模	4
2.2 P0-P5 流水线	4
3 模块分工与课程知识点	6
3.1 五个工作包划分	6
3.2 课程知识点对应	6
3.3 核心成果概览	10
4 P1: 二维候选路径生成	11
4.1 建模对象与输出目标	11
4.2 算法框架	12
4.3 Grid A*: 栅格可达性基线	12
4.4 VG: 可视图几何最短路	13
4.5 AVG: 加入转角偏好的增强可视图	13
4.6 DAVG: 动态风险代价与活跃可视图	14
4.7 算法对比实验设计	15
4.8 代表性 OD 对路径形态比较	16
4.9 mixed 多候选路径库	17
4.10 输出接口	18
4.11 与后续模块的关系	18
5 P2: 任务分配与任务排序模型	19
5.1 输入、输出与决策内容	19
5.2 P2 数学模型	19
5.3 目标规划口径	20
5.4 求解方法实现	21
6 P3: 时空轨迹展开与冲突修复	22
6.1 模块定位与输入输出	22
6.2 轨迹展开模型	23
6.3 冲突检测模型	24
6.4 局部修复策略	25

6.5	目标函数与选择规则	25
6.6	可视化结果与轨迹解释	26
6.7	输出文件与复现实验检查	27
6.8	P4 接口与工程可解释性	29
6.9	方法局限与全局最优讨论	29
7	P4: 动态事件与滚动时域重排	30
7.1	模块定位与输入输出	30
7.2	图形化运行过程	31
7.3	动态事件建模	33
7.4	滚动时域集合划分	34
7.5	混合整数规划抽象	35
7.6	字典序目标规划	37
7.7	求解策略与实现流程	37
7.8	与 P2/P3 的区别	38
8	P5: 灵敏度分析	39
8.1	基线校验与占用热点识别	39
8.2	AMR 数量的资源边际分析	40
8.3	任务负荷与系统容量边界	42
8.4	瓶颈空间簇与容量影子价值	44
8.5	Morris 与 Sobol 全局灵敏度筛选	47
9	实验结果与分析	49
9.1	路径源对调度结果的影响	49
9.2	P2 方法对比	49
9.3	P2 敏感性分析	50
9.4	P3 冲突修复结果	51
9.5	P4 动态滚动重排结果	52
9.6	动态事件影响范围	53
9.7	重排任务明细	55
9.8	可行性验证	56
9.9	目标函数指标解释	56
9.10	与中间候选方案的比较	58
9.11	图形结果解释	59
9.12	适用范围与扩展实验	59
10	结论	60
A	成员分工	62

摘要

本项目面向智能仓储中多台自主移动机器人（AMR）的取送货调度问题，围绕任务分配、路径选择、时间窗、电量约束、轨迹冲突和动态扰动重排建立一套分阶段优化模型。全文按五个工作包展开：P1 生成二维候选路径库，P2 完成任务分配与排序，P3 展开时空轨迹并修复冲突，P4 在通道封锁、AMR 延误和新增任务下进行滚动时域重排，P5 沿用前序调度链路开展灵敏度分析和管理解释。

模型层面，本项目将多 AMR 调度抽象为带候选路径选择的取送货调度问题。P1 将二维地图上的路径搜索结果转化为成本表和轨迹表；P2 使用字典序目标规划处理可行性、服务等级、完工时间、空驶成本、负载均衡和能耗；P3 将 P2 的任务表展开为二维轨迹样本，检测 footprint 冲突和节点容量冲突，并通过等待、换路 and 同车相邻换序进行修复；P4 在动态事件到达后冻结已执行任务，对未来任务池进行后悔值插入、局部搜索、候选路径重评估和冲突等待修复；P5 围绕 AMR 数量、任务负荷和瓶颈通道容量开展局部边际分析与全局灵敏度筛选。

实验在 4 台 AMR、12 个静态任务和 1 个动态新增任务的仓库算例上完成验证。P2 输出的静态计划无缺失路径、无电量阈值违反和无延期；P3 将 21 个初始时空冲突修复为 0，最终 $C_{\max} = 46.59697$ ；P4 在 3 个动态事件下保持剩余轨迹冲突、封锁区违规和 AMR 延误违规均为 0，并将新增任务插入到 AMR4 的未来任务序列中，最终滚动重排方案的总延期为 23.405971， $C_{\max} = 61.918082$ 。P5 进一步表明：4 台 AMR 是当前算例的服务阈值，固定 4 台 AMR 时保守任务容量边界为 14 个任务，C03 是边界工况下的主导空间瓶颈；Morris 与 Sobol 结果共同指向 AMR 数量为 $C_{\max}$ 的主导因素，其次为任务负荷。

1 问题背景与实现目标

1.1 现实背景

智能仓储中的 AMR 需要在货架区、分拣台、出库口和充电点之间执行取送货任务。实际系统中，多台机器人同时共享二维通道、路口和作业点。若只给每台机器人独立规划最短路径，容易出现任务负载不均、空驶距离过长、节点同时占用、狭窄通道会车冲突、动态封锁后大范围延误等问题。

因此，本文关注的核心问题可以表述为：在给定仓库二维平面、AMR 状态、任务时间窗、候选路径和动态事件的条件下，如何决定任务分配、任务顺序、路径选择和执行时间，使调度方案满足路径可达、电量安全、时空无冲突和动态事件约束，并尽量降低延期、完工时间、空驶成本和扰动范围。

1.2 研究目标

本项目的研究目标包括：

1. 构造二维仓库场景，统一节点、障碍物、货架区、封锁区和动态事件的数据接口。
2. 用 A*、VG、AVG、DAVG 和 mixed 候选路径库为 P2–P4 提供路径成本和轨迹样本。
3. 在 P2 中比较贪心、两阶段整数规划、联合 MILP 目标规划和后悔值插入局部搜索四类方法。
4. 在 P3 中把任务级计划展开为 0.1 秒粒度轨迹，检测并修复 AMR footprint 冲突和节点容量冲突。
5. 在 P4 中处理通道封锁、AMR 延误和新增任务，形成动态滚动重排结果。
6. 在 P5 中开展 AMR 数量、任务负荷、瓶颈通道容量与 Morris/Sobol 全局灵敏度分析，形成运营解释。
7. 输出可复现实验 CSV、甘特图、路径图、敏感性分析图和动态演示 GIF。

2 系统数据与模块接口

2.1 二维仓库建模

本文统一采用“二维仓库平面图 + 障碍物 + 轨迹采样”的建模口径。原始数据包括：

文件	含义
<code>nodes.csv</code>	AMR 初始点、取货点、分拣点、出库点、充电点的二维坐标
<code>amrs.csv</code>	AMR 初始位置、电量和速度系数
<code>tasks.csv</code>	静态任务的取货点、送货点、服务时间、时间窗和优先级
<code>floor_zones.csv</code>	仓库功能区，例如货架区、通道区和分拣区
<code>floor_obstacles.csv</code>	墙体、货架块和动态风险区域
<code>dynamic_events.csv</code>	<code>area_block</code> 、 <code>amr_delay</code> 、 <code>new_task</code> 三类动态事件

表 1: 原始数据接口

表 1 给出了本文建模所需的基础数据，图 1 则将这些数据转化为可视化仓库场景。后续 P1 的路径搜索、P2 的任务调度和 P3/P4 的轨迹检测都以该二维场景为统一空间基准。

2.2 P0–P5 流水线

本文采用如下分阶段流水线：

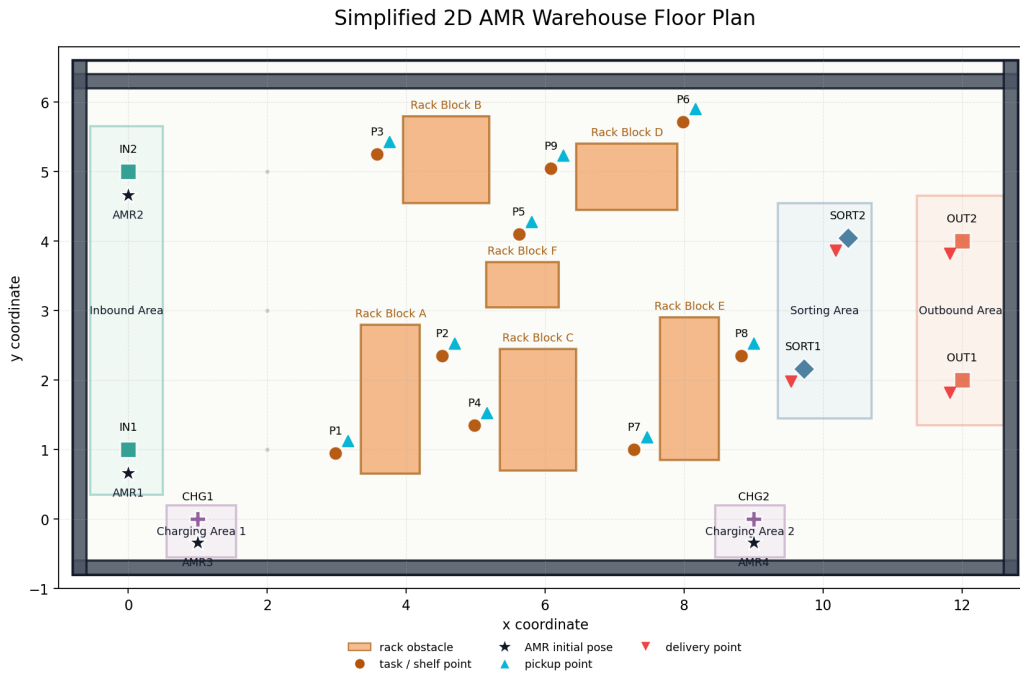


图 1: P0 生成的二维仓库平面图

1. P0: 读取仓库区域、节点和障碍物, 输出 `outputs/p0/warehouse_floor_plan.png`。
2. P1: 为关键节点之间生成候选路径、路径成本矩阵和轨迹采样表。
3. P2: 读取 P1 路径成本, 输出任务分配和 AMR 内部任务序列。
4. P3: 读取 P2 任务表和 P1 轨迹样本, 展开时空轨迹并修复冲突。
5. P4: 读取 P3 无冲突基准计划和动态事件, 执行滚动时域重排。
6. P5: 沿用 P1-P4 的调度链路和评价口径, 开展基线校验、资源边际、任务容量、瓶颈簇容量和全局灵敏度分析。

主流程命令如下:

```

1 python .\src\p0_data_scene\plot_floor_plan.py
2 python .\src\p1_candidate_paths\generate_candidate_paths.py --
  algorithm basic_astar
3 python .\src\p2_assignment\build_mixed_paths.py
4 python .\src\p2_assignment\assign_and_sequence_tasks.py --
  method m2 --p1-algorithm mixed
5 python .\src\p3_schedule_conflicts\
  schedule_and_detect_conflicts.py --p1-algorithm mixed
6 python .\src\p4_dynamic_reschedule\dynamic_reschedule.py
7 python .\src\p4_dynamic_reschedule\plot_p4_results.py
  
```

3 模块分工与课程知识点

3.1 五个工作包划分

为了体现多成员协作和完整建模链路，本文按 P0/P1、P2、P3、P4、P5 五个工作包组织报告结构。每个工作包既有清晰的输入输出边界，也对应运筹学课程中的不同知识点。

负责人	工作包	主题	主要任务	交付内容
李彦霖	P0/P1	场景与路径生成	构造二维仓库平面图，建立障碍物与关键节点数据；比较 A*、VG、AVG、DAVG，并形成 mixed 多候选路径接口	场景图、路径库、轨迹样本
朱熠正	P2	任务分配与排序	决定任务指派、AMR 内部任务顺序、空驶/载货路径选择，并校验时间窗与电量	任务分配表、方法对比
江亚楠	P3	时空冲突修复	将任务级计划展开为轨迹级计划，检测 footprint 冲突和节点容量冲突，并通过等待、换路、换序修复	修复后时间表、冲突日志
倪振昊	P4	动态滚动重排	处理通道封锁、AMR 延误和新增任务，冻结历史任务并重排未来任务池	动态重排表、可视化
张铭滔	P5	灵敏度分析	沿用 P1-P4 的调度链路和评价口径，形成基线校验、边际分析、容量边界、瓶颈簇验证和全局灵敏度筛选	灵敏度结果表、灵敏度图表、管理建议

表 2: 五个工作包与小组分工

表 2 展示了五个工作包之间的边界：P0/P1 负责场景和路径，P2 负责任务级调度，P3 负责轨迹级可执行性，P4 负责动态扰动，P5 负责灵敏度分析和管理解释。这样的拆分能够对应五人分工，同时保证各模块之间通过明确数据接口串联。

3.2 课程知识点对应

3.2.1 P1：图论最短路与候选路径生成

P1 对应图论和最短路思想。基础 A* 在二维栅格上搜索可行路径，VG 将仓库障碍物转化为可视图最短路问题，AVG 在路径长度之外加入转弯代价，DAVG 进一步把动态风险区域转化为路径代价。mixed 路径库把多种路径源融合为多候选集合，使“路径选择权”从底层寻路上升到调度层，体现了图论最短路与上层组合优化之间的衔接。

3.2.2 P2：任务分配中的整数规划、目标规划与求解方法

P2 是课程知识点最集中的部分。任务指派变量 x_{ki} 、首任务变量 s_{ki} 、任务顺序变量 u_{kij} 和路径选择决策共同构成 0-1 混合整数规划。每个任务必须且只能分配给一台 AMR，每台 AMR 内部任务必须形成一条线性任务链，时间衔接和电量链约束保证调度结果在任务级别可执行。

0-1 整数规划。 P2 的任务指派、任务排序和路径选择都属于典型离散决策。 x_{ki} 决定任务归属, s_{ki} 决定每台 AMR 的首任务, u_{kij} 决定同一 AMR 内任务之间的前后继关系。三类 0-1 变量共同描述“谁执行、先执行谁、走哪条路”, 对应整数规划中用二进制变量表达组合决策的基本方法。

链式序列与时间衔接。 P2 不仅要求每个任务被分配, 还要求每台 AMR 的任务形成一条从初始点出发的线性链。时间递推约束保证后继任务必须在前序任务完成并空驶到取货点之后才能开始, 最早开始时间和最晚完成时间则分别体现硬时间窗和延期软约束。

目标规划。 在目标函数上, P2 采用目标规划的优先级因子思想, 并避免把所有目标压缩为简单加权和。模型先压低缺失路径和电量违反, 再处理高优先级延期任务数、延期任务数、时间效率 F_2 和运行成本 F_3 。这种 $P_1 \gg P_2 \gg P_3$ 的字典序结构能够避免“用较低路径成本抵消高优先级任务延期”这类不合理结果。

分支定界与割平面。 P2 的求解方法也对应多个课程章节。m1 的阶段一是 0-1 指派模型, 可用分支定界求解; 阶段二对每台 AMR 的车内任务排序采用 Held-Karp 型动态规划, 状态为“已完成任务集合 + 当前末任务”。m2 将指派、排序、时间和电量统一写入联合 MILP, 并通过分支定界和割平面思想求解。m3 使用 regret-2 后悔值插入构造初始解, 再通过 relocate、swap、reroute 等邻域搜索逼近局部最优解, 体现了启发式算法在较大规模调度中的工程价值。

动态规划与启发式优化。 m1 的车内排序子问题可以看作动态规划: 状态记录已完成任务集合和当前末任务, 转移时尝试追加下一个任务并比较字典序评价。m3 则代表启发式优化思路, 先用 regret-2 插入构造高质量初始解, 再通过 relocate、swap、reroute 等邻域搜索降低目标值。这一组方法使 P2 同时具备“可解释的精确模型”和“可扩展的快速求解器”。

3.2.3 P3: 时空网络、容量约束与资源受限调度

P3 对应的核心运筹学问题, 是在已给定任务分配和任务顺序后, 进一步判断多台 AMR 在时间和空间上的资源占用是否可行。P2 更偏向任务层面的整数规划, P3 则把计划放入“时间-空间-资源”框架中检查, 重点体现时空网络建模、容量约束、互斥约束、资源受限调度和启发式修复等知识点。

时空网络建模。 时间扩展网络是运筹学中处理动态路径和动态占用问题的常用方法。它把普通网络中的节点和边复制到不同时间层, 使“车辆在什么时刻位于什么位置”能够被显式表示。P3 将 P2 的任务级计划展开为带绝对时间的二维轨迹, 本质上就是构造离散时间下的时空网络占用表。这样, 路径不再只是起点到终点的几何连接, 而是变成 AMR 在各个时刻对仓库空间资源的连续占用。

容量约束。 容量约束用于限制同一资源在同一时刻能够被多少个对象占用。在 P3 中，取货点、分拣点、出库点等关键节点可视为网络资源，其容量通常为 1，可写成

$$\sum_{k \in K} y_{kvt} \leq c_v.$$

其中 y_{kvt} 表示 AMR k 在时刻 t 是否占用节点 v ， c_v 表示节点容量。P3 对 P*、SORT*、OUT* 等节点的占用检测，就是这一类网络容量约束在仓储调度中的体现。

互斥约束与安全距离约束。 多 AMR 运行还需要满足空间互斥条件，即两台机器人不能在同一时间占用过近的空间。P3 用 footprint 半径和安全裕量构造距离约束：

$$\|X_i(t) - X_j(t)\| \geq \rho_i + \rho_j + \sigma, \quad i \neq j.$$

这属于运筹学中常见的冲突避免约束或互斥资源约束。与只限制图上同一节点、同一边不同，P3 直接在二维坐标上判断 AMR 间距，因此能够反映仓库通道宽度、货架间距和机器人 footprint 对可执行性的影响。

资源受限制度。 从调度理论看，P3 可理解为资源受限制度问题。AMR 是执行任务的机器资源，仓库节点和通道是共享空间资源，空驶、取货、载货和等待都是时间轴上的作业活动。P2 给出的任务顺序只是初始作业序列，P3 需要检查这些作业活动叠加后是否超出共享资源容量。若发生冲突，就说明任务级计划虽然可行，但在资源受限的执行层面仍不可行。

启发式算法与局部搜索。 P3 没有重新求解全局任务分配模型，而是在 P2 计划附近进行局部修复。等待、换路和同车相邻换序分别对应时间调整、路径邻域搜索和序列邻域搜索。算法每轮选择当前冲突，生成有限候选动作，再比较修复后的冲突数和目标值。这体现了启发式算法在大规模组合调度中的思想：不追求一次性建立过大的全局精确模型，而是利用局部搜索快速得到可执行方案。

目标规划与字典序优化。 P3 的评价规则体现目标规划思想。剩余冲突数具有最高优先级，因为存在冲突的方案不能执行；在冲突数相同的情况下，再比较延期、最大完工时间 $C_{\max}$ 、新增等待和对原计划的扰动。这样的字典序结构避免了用较短完工时间去交换安全冲突，也避免为了消除局部冲突而过度破坏 P2 的任务安排。最终，P3 将任务级计划转化为满足容量约束和安全互斥约束的轨迹级可执行计划。

3.2.4 P4：动态规划思想、混合整数规划、目标规划与滚动时域重排

P4 对应动态规划中的“状态更新后继续决策”思想，也把 P2/P3 的静态调度模型推广为带扰动事件的动态取送货时间窗问题。动态事件发生后，系统以事件时刻为滚动边界，将任务划分为固定任务集 J_{fix} 和可重排任务集 J_{free} ：已经完成、正在执行且不受

事件影响的任务保持不变，受临时新增任务、通道封锁、AMR 延误和局部排程调整影响的未来任务进入重优化窗口。这里的关键不是重新从零开始，而是把“事件发生后的 AMR 可用时间、当前位置、剩余任务池”视为新的系统状态，再在这个新状态上继续安排后续决策。这与动态规划中“状态转移后继续求解剩余子问题”的思路是一致的。

混合整数规划建模。 从运筹学模型看，P4 可抽象为带动态事件约束的混合整数规划。二进制变量 x_{ki} 表示任务 i 是否由 AMR k 执行， y_{kij} 表示同一 AMR 内任务 i 与任务 j 的前后继关系，路径选择变量表示 P1 候选路径库中的具体路径；连续变量 S_i, C_i 表示任务开始和完成时间。模型约束包括任务唯一分配、AMR 序列连续、时间递推、路径可达性、电量安全、通道封锁时间窗避让、AMR 延误窗口互斥，以及 P3 给出的轨迹 footprint 冲突约束。因此 P4 在动态约束下联合决定“谁执行、何时执行、走哪条路径、是否等待”。

目标规划与字典序优化。 P4 的目标函数采用目标规划和字典序多目标思想。第一层目标是硬可行性，优先使轨迹冲突数、封锁违规数、AMR 延误违规数、缺失路径数和电量违反数为 0；第二层目标是服务水平，最小化新增延期、总延期和高优先级任务延期；第三层目标是运行质量，控制最大完成时间、路径成本、能耗、负载不均衡和被扰动任务数。这样的目标层级避免了用较小运行成本去抵消硬约束违规，也避免了“全局重排虽然目标值低但扰动过大”的不合理方案。

滚动时域启发式求解。 完整时空 MILP 的变量规模会随 AMR 数、任务数、候选路径数和时间离散粒度快速增长。P4 采用滚动时域启发式：冻结历史计划，构造受影响任务池，通过后悔值插入、冲突感知短名单、有界局部修复和等待调整快速生成可行解。若用课程语言概括，这一过程可以理解为“先根据事件完成一次状态转移，再在新状态上继续选择后续动作”的动态规划思想，加上整数规划建模与目标规划评价。它的核心价值是在实时性、可行性、服务水平和计划稳定性之间取得可解释的折中。

3.2.5 P5：灵敏度分析

P5 对应运筹学中的**灵敏度分析、容量规划和决策支持**。在 P1–P4 已形成可行调度链路的基础上，P5 进一步研究关键参数变化对目标函数 $C_{\max}$ 、总延期和可行性的影响。由于 AMR 数量 m 、任务量 n 和瓶颈容量 q_C 均为离散变量，P5 采用**情景重优化 + 有限差分 + 方差分解**的方法，对边际收益和主导影响因素进行定量评估。

离散资源边际分析。 P5 将 AMR 数量视为整数资源变量，对不同 m 逐点重新求解调度问题，并用前向差分衡量增加 1 台 AMR 对系统性能的边际改善。这对应运筹学中资源配置变量的边际分析。若 $\Delta C_{\max}$ 随 m 增加而快速减小，说明继续增加 AMR 的边际收益递减，系统逐渐进入资源相对充足状态；若改善幅度仍然明显，则说明 AMR 数量仍是限制系统性能的重要资源约束。

系统容量边界与可行性判定。 任务负荷分析对应**容量规划**问题。P5 在同一任务池上构造嵌套前缀，并对不同任务规模逐点重优化；同时结合最晚完成时刻、延期任务数、路径缺失和电量违规构造可行性判据，再通过 all-seeds 规则确定保守容量边界。该过程体现了**资源受限调度**和**可行域判定**思想，即先判定系统在给定任务负荷下能否稳定完成服务，再进一步评价服务质量和性能水平。

瓶颈容量与影子价值。 对局部通道簇容量 q_C 的放松实验，对应**离散影子价格分析**。当某一瓶颈通道的容量增加 1 个单位后，重新执行冲突修复与调度求解，进而观察 $C_{\max}$ 和总等待时间的下降量。该下降量可用于衡量局部通道容量的边际价值，并为瓶颈识别、通道扩容和资源优先配置提供依据。

Morris 与 Sobol 全局灵敏度。 P5 的全局筛选使用 Morris 和 Sobol 两类经典灵敏度分析方法。Morris 方法通过基本效应 μ^* 和波动度 σ 筛选主导因素，适用于离散、非线性和具有局部交互的系统；Sobol 方法通过方差分解计算一阶效应和总效应，用于区分单因素主效应与多因素交互效应。两者结合后，可以同时识别关键参数及其交互影响，从而提高 P5 对调度系统性能变化来源的解释能力。

3.3 核心成果概览

为了突出模型链路的实际效果，本文将最关键的实验结论概括如下：

成果点	关键结果	含义
P1 多候选路径库	mixed 平均每个 OD 有 1.77 条候选，最多 4 条	调度层不再被单一路径源绑定，可在多路径中择优
P2 联合优化	m2/m3 将 $C_{\max}$ 从 m0 的 49.015 降至 40.578	任务指派与排序联合建模显著改善系统完工时间
P2 快速启发式	m3 与 m2 达到相同 F2/F3，但 0.198s 对 116.728s	启发式在本算例达到精确模型质量，速度提升约 589 倍
P3 冲突消解	初始 21 个时空冲突全部修复为 0	证明任务级可行不等于轨迹级可执行，P3 层不可省略
P4 动态重排	轨迹冲突、封锁违规、AMR 延误违规均为 0	动态扰动下仍能保持计划可执行性
P5 灵敏度分析	识别 4 台 AMR 服务阈值、14 个任务保守容量边界、C03 主导空间瓶颈，并通过 Morris/Sobol 验证 AMR 数量为主导因素	将调度结果转化为扩车、控量和通道改造的决策依据

表 3: 核心成果概览

表 3 的作用是把后文各模块的实验证据先集中呈现。本文的贡献体现在从路径候选、任务优化、轨迹冲突修复、动态重排到灵敏度分析的完整闭环：P1 提供路径选择空间，P2 给出高质量任务级计划，P3 保证轨迹级可执行，P4 验证动态扰动下的鲁棒性，P5 识别资源阈值、容量边界和主导瓶颈。

4 P1：二维候选路径生成

P1 是整个系统的路径基础层。它不直接决定任务由谁执行，也不处理多机器人避碰，而是在二维仓库平面上为关键节点对生成候选路径、路径成本、栅格轨迹和轨迹采样表。下游 P2 使用 P1 的成本表做任务分配与排序，P3/P4 使用 P1 的轨迹样本展开二维时空轨迹。因此，P1 的核心作用不是简单寻找一条最短路，而是把仓库平面、障碍物、安全膨胀、路径偏好和动态风险转化为后续调度模型可读取的结构化输入。

4.1 建模对象与输出目标

P1 的输入来自 P0 整理后的二维仓库场景，主要包括节点表、区域表、障碍物表、AMR 初始位置和任务取送货点。关键节点集合记为 V_K ，其中包含入库口、出库口、货架点、分拣台、充电点和 AMR 初始点。P1 对所有有序节点对 $(i, j), i \neq j$ 生成路径，因此每一种单算法在本算例中输出 $17 \times 16 = 272$ 条可行路径。

从数学上看，P1 需要为每个 OD 对输出一条折线路径

$$p_{ij} = (q_0, q_1, \dots, q_m), \quad q_0 = i, \quad q_m = j,$$

其中每个 $q_\ell = (x_\ell, y_\ell)$ 是二维平面上的路径点。路径的几何距离为

$$d(p_{ij}) = \sum_{\ell=0}^{m-1} \|q_{\ell+1} - q_\ell\|_2,$$

在本文实现中 AMR 标准速度取 $v = 1.0$ ，所以物理通行时间为

$$t(p_{ij}) = \frac{d(p_{ij})}{v}.$$

除了距离和时间，P1 还记录转角数、转角代价、动态区域代价、轨迹采样点数等指标。这些指标一方面用于算法比较，另一方面进入 P2 的路径选择和 P3/P4 的轨迹展开。

障碍物处理采用“AMR footprint 膨胀”思想。货架、墙体和动态封锁区在路径规划时不是只按原始几何边界处理，而是按机器人半径进行膨胀，使路径几何中心线与障碍物保持安全距离。这样，P1 输出的路径在后续展开为 footprint 轨迹时才不会天然穿越障碍区域。

4.2 算法框架

P1 实现了 `basic_astar`、`vg`、`avg`、`davg` 四类路径生成算法，并进一步通过 `mixed` 路径库形成多候选接口。四类单算法分别代表不同路径偏好：栅格可达性、几何短路、转弯平滑性和动态风险规避。

算法	核心思想	当前实现中的作用
<code>basic_astar</code>	将仓库离散为 0.1m 栅格，在 8 邻域上用 A* 最小化几何距离	作为最朴素的可达性基线，路径贴合栅格方向，通常折点较多
<code>vg</code>	构造障碍物膨胀边界的可视图，在可见顶点之间搜索最短折线	作为连续几何最短路基线，路径通常更短、更少折点
<code>avg</code>	在 VG 基础上把上一顶点纳入搜索状态，对转角大小加惩罚	倾向选择转向更少、更平滑的路径，体现运动执行偏好
<code>davg</code>	在 AVG 基础上加入动态风险区域暴露代价，并采用局部活跃可视图	用于动态封锁或风险场景，倾向绕开临时风险区域
<code>mixed</code>	合并四类算法输出，并按几何去重保留多候选	为 P2/P3/P4 提供同一 OD 下多路径可选空间

表 4: P1 路径算法在最终系统中的定位

表 4 说明 P1 并不是简单比较几种最短路算法，而是为后续调度层构造不同偏好的候选路径。A* 偏重栅格可达性，VG 偏重连续空间中的几何最短，AVG 引入转弯平滑性，DAVG 引入动态风险代价，mixed 则把这些不同路径偏好融合成调度层可选择的路径库。

4.3 Grid A*: 栅格可达性基线

`basic_astar` 将二维仓库转化为分辨率 0.1m 的规则栅格。每个栅格单元用其中心点代表空间位置，若中心点落入膨胀后的硬障碍物，则该单元标记为不可通行。搜索动作采用 8 邻域：

$$\Delta = \{(-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (-1, 1), (1, -1), (1, 1)\}.$$

横纵向移动代价为一个栅格宽度，斜向移动代价为 $\sqrt{2}$ 个栅格宽度。A* 的评价函数为

$$f(n) = g(n) + h(n),$$

其中 $g(n)$ 是从起点到当前栅格的累计距离， $h(n)$ 是当前栅格到终点的欧氏直线距离。由于 8 邻域移动的真实最短剩余距离不会小于欧氏距离，这个启发函数是可接受的，不会高估剩余代价。

实现中还加入了两个工程细节。第一，如果起点或终点恰好位于膨胀障碍物内，会在局部半径内寻找最近可通行栅格，避免由于节点坐标贴近货架边界而直接失败。第二，斜向移动时检查相邻两个正交单元，禁止“贴角穿越”，防止路径从两个障碍物角点之间不合理穿过。

Grid A* 的优点是直观、稳定、容易处理任意形状的占用区域；缺点是路径受栅格方向限制，可能产生较多方向变化。本文只把转角数作为报告指标，不把转角惩罚加入 A* 搜索目标，因此它是一个纯距离基线。后续的 VG/AVG/DAVG 都可以与该基线对比，说明连续几何建图和路径偏好项的价值。

4.4 VG：可视图几何最短路

vg 不再在密集栅格上扩展，而是在连续平面中构造稀疏可视图。可视图的顶点包括起点、终点和膨胀障碍物的角点。若两个顶点之间的线段不穿过任何硬障碍物内部，则二者之间连一条边，边权为欧氏距离。随后在这个可视图上运行 A*，启发函数仍是到终点的直线距离。

VG 的关键是把“绕障碍物”问题转化为“在可见角点之间选折线”的图搜索问题。若不考虑转弯和动态风险，二维多边形障碍物环境中的最短折线通常会贴近障碍物膨胀边界的顶点。因此，VG 比 Grid A* 更接近连续空间中的几何最短路，路径折点也少得多。

本文实现中，线段可见性使用矩形裁剪思想判断线段是否穿过障碍物内部。触碰边界被允许，因为障碍物已经经过安全膨胀；真正被拒绝的是线段与膨胀障碍物内部有正长度重叠的情况。墙体作为仓库边界，不参与候选绕行角点，避免路径沿外墙边界产生无意义顶点。

VG 的输出中 `turn_cost`、`dynamic_cost` 均为 0，`total_cost` 等于几何距离。它在 P1 里提供的是“最短可见折线”的参照物：如果某条路径在 VG 中已经很短且折点很少，说明该 OD 对的空间约束比较直接；如果与 A* 差异很大，则说明栅格离散或障碍边界对路径形态影响明显。

4.5 AVG：加入转角偏好的增强可视图

普通 VG 只关心路径长度，可能选择一条距离略短但转弯较急的路线。对于 AMR 来说，频繁或剧烈转向会带来控制代价、速度损失和执行不稳定性。AVG 在 VG 的几何图基础上引入转角惩罚，目标函数变为

$$c_{\text{AVG}}(p) = d(p) + \lambda_{\theta} \sum_{\ell=1}^{m-1} \theta_{\ell},$$

其中 θ_{ℓ} 是路径在顶点 q_{ℓ} 的航向变化角， $\lambda_{\theta} = 0.5$ 为单位弧度转角代价。

为了在搜索过程中正确计算转角，AVG 的状态不再只是当前顶点，而是

$$s = (q_{\text{prev}}, q_{\text{cur}}).$$

当搜索从 q_{cur} 扩展到 q_{next} 时, 就可以根据上一段和下一段的航向计算当前顶点处的转角代价。该状态扩展使 A^* 的转移代价从单纯边长变为

$$c(s, q_{\text{next}}) = \|q_{\text{next}} - q_{\text{cur}}\|_2 + \lambda_\theta \Delta\phi(q_{\text{prev}}, q_{\text{cur}}, q_{\text{next}}).$$

第一步没有上一段航向, 因此不收取转角代价; 同时实现中跳过立即掉头的扩展, 减少无意义循环。

需要强调的是, AVG 中的转角代价不是物理通行时间。本文仍用几何距离除以速度计算 `travel_time`, 转角项只进入 `total_cost`, 用于表达路径偏好。这样做的好处是保持时间窗模型简洁, 同时让 P2 在路径选择时可以区分“距离短但转弯多”和“略长但更平顺”的路线。

4.6 DAVG: 动态风险代价与活跃可视图

DAVG 在 AVG 的基础上处理动态封锁或风险区域。它保留状态增强和转角惩罚, 同时对靠近动态区域的可视边加入暴露代价:

$$c_{\text{DAVG}}(p) = d(p) + \lambda_\theta \sum_{\ell=1}^{m-1} \theta_\ell + \sum_{e \in p} c_{\text{dyn}}(e).$$

对于一条边 e , 若它到动态区域的最小距离为 δ_e , 动态风险缓冲半径为 $B = 1.0$, 权重为 $\lambda_d = 2.0$, 则实现中的动态代价可理解为

$$c_{\text{dyn}}(e) = \begin{cases} \lambda_d \left(1 - \frac{\delta_e}{B}\right) |e|, & \delta_e < B, \\ 0, & \delta_e \geq B. \end{cases}$$

这意味着边越长、越靠近动态区域, 付出的风险代价越大。若路径远离动态区域, 则 DAVG 会退化为 AVG。

DAVG 还加入活跃区域建图: 对每个 OD 对, 只把起终点走廊附近的障碍物角点加入候选顶点, 同时动态障碍物角点始终保留。这样可以减少远离当前 OD 的障碍角点带来的图规模。为了不牺牲可行性, 可见性检测仍对所有硬障碍物进行; 若局部活跃图无法找到路径, 则回退到完整障碍角点图。

在当前静态主算例中, DAVG 的平均动态代价为 0, 说明选出的路径没有进入动态风险缓冲区。因此, DAVG 与 AVG 的几何距离、转角数和总成本在 P1 层面基本一致。这不是 DAVG 失效, 而是说明当前 P1 静态路径比较没有触发风险绕行; DAVG 的主要价值体现在后续 P4 动态事件或封锁场景中。

4.7 算法对比实验设计

为支撑报告中的 P1 横向比较，本文新增脚本 `run_p1_experiments.py`，位于 `src/p1_candidate_paths/`。该脚本只读取已有的 P1 处理结果，不覆盖任何原始结果；所有新增实验产物写入 `outputs/p1_experiments/`。输出包括算法总体指标表、代表性 OD 对路径对比图、mixed 候选数量热力图 and 对应 CSV。

算法	路径数	平均距离	平均转角数	平均动态代价	平均总成本
Grid A*	272	6.463	9.206	0.000	6.463
VG	272	6.054	1.846	0.000	6.054
AVG	272	6.071	1.765	0.000	6.695
DAVG	272	6.071	1.765	0.000	6.695

表 5: P1 四类单算法总体指标对比

表 5 体现了不同算法的主要差异。Grid A* 的平均距离为 6.463，高于 VG 的 6.054；平均转角数为 9.206，也显著高于 VG 的 1.846。这说明栅格离散会引入阶梯状路径和更多方向变化。VG 在连续可视图上寻找几何短路，所以距离和折点都更少。AVG 的平均距离略高于 VG，但平均转角数进一步降低到 1.765，说明转角惩罚会让搜索在少量距离损失下获得更平滑的路径。DAVG 在当前静态算例中与 AVG 一致，因为动态风险代价没有被触发。

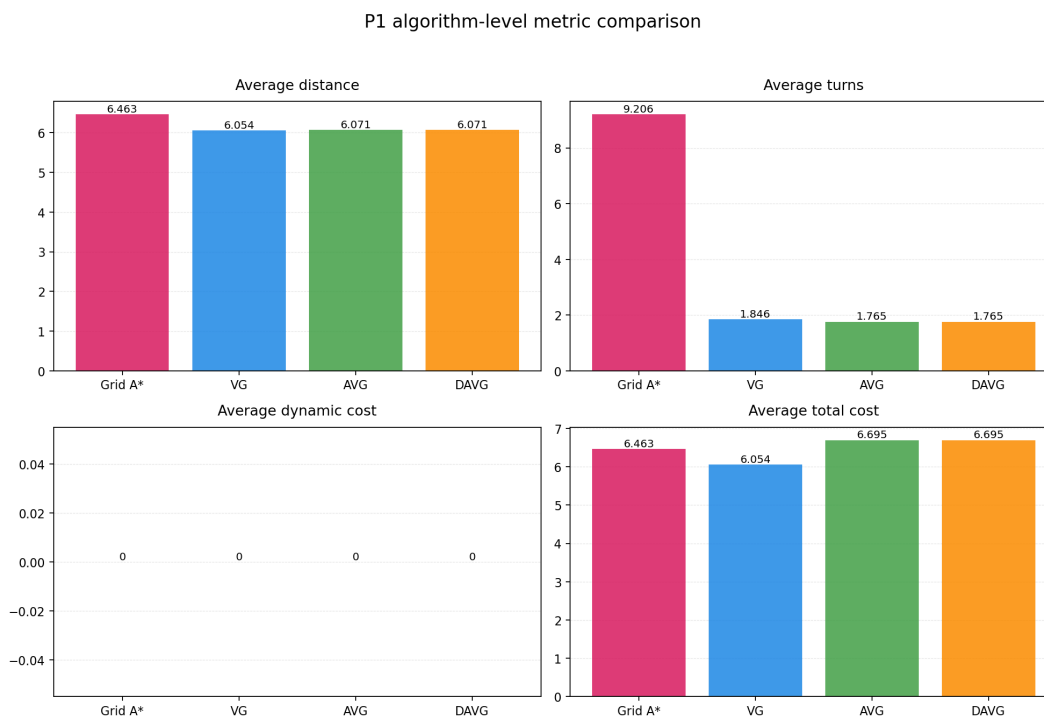


图 2: P1 四类算法在距离、转角、动态代价和总成本上的总体对比

图 2 从可视化角度进一步说明：如果只看距离，VG 是当前算例中最短的单路径源；如果考虑转向平滑性，AVG/DAVG 比 VG 稍微牺牲距离，但用转角惩罚表达执行偏好；

如果考虑动态风险，当前样本中四类路径动态代价均为 0，说明风险区域并未影响这些静态 OD 最优路径。这为后文解释 P2 中不同路径源对调度结果的影响提供了基础。

4.8 代表性 OD 对路径形态比较

为了更直观地解释四类算法的路径差异，本文选取 IN1 → P5 作为路径图案例。该 OD 对需要从入库区进入货架中部，路径需要绕开货架 A/F，能够展示栅格折线、可视图短路和转角代价评价之间的区别。另选 P8 → OUT2 作为表格补充，用于说明接近直线通行时栅格算法仍可能产生额外转向。

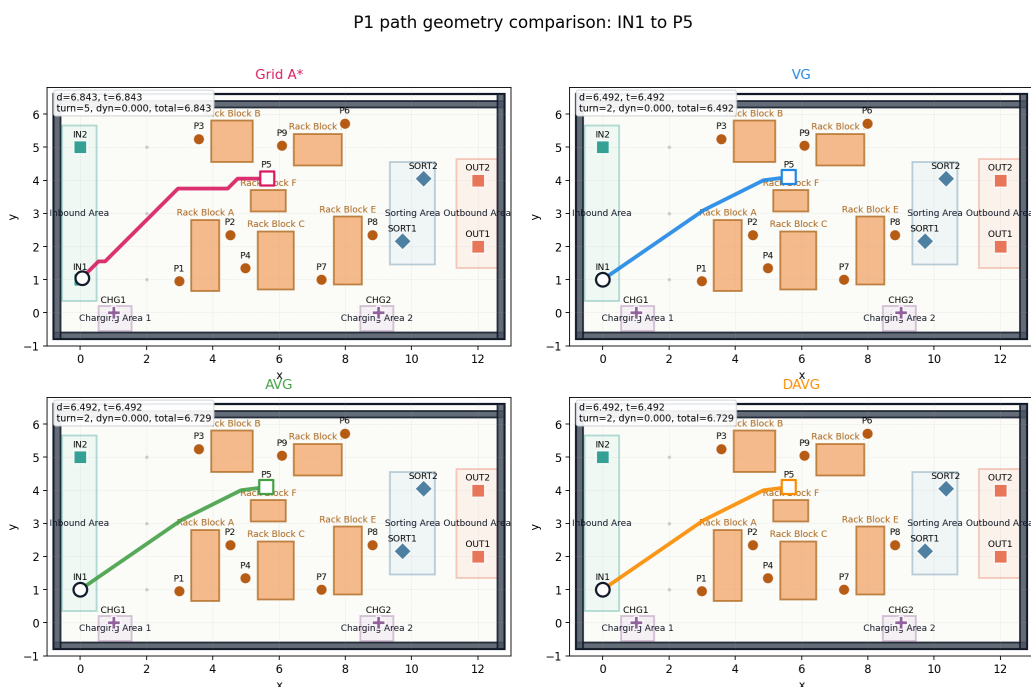


图 3: IN1 → P5 的四类 P1 路径形态对比

图 3 中，Grid A* 路径沿栅格方向逐步上行并右移，距离为 6.843，转角数为 5；VG 直接利用可视边绕过货架，距离降至 6.492，转角数为 2。AVG 与 DAVG 在这个 OD 上选择了和 VG 相同的几何折线，但由于 AVG/DAVG 的 `total_cost` 中包含转角惩罚，其总成本为 6.729，高于 VG 的 6.492。这说明同一条几何路径在不同算法的评价口径下可能有不同总成本，P2 在读入路径源时需要明确使用的是 `travel_time` 还是 `total_cost`。

作为补充，P8 → OUT2 在连续空间中几乎可以直线到达，因此 VG、AVG、DAVG 均输出距离 3.583、转角数 0 的直线路径。Grid A* 由于受到 0.1m 栅格和 8 邻域运动方向限制，输出距离 3.863、转角数 18 的折线。这个案例说明，即使起终点之间没有复杂绕障碍需求，栅格路径也可能因为离散化而产生额外转向；连续可视图算法可以显著改善路径几何质量。

OD 对	算法	距离	转角数	动态代价	总成本
IN1->P5	Grid A*	6.843	5	0.000	6.843
IN1->P5	VG	6.492	2	0.000	6.492
IN1->P5	AVG	6.492	2	0.000	6.729
IN1->P5	DAVG	6.492	2	0.000	6.729
P8->OUT2	Grid A*	3.863	18	0.000	3.863
P8->OUT2	VG	3.583	0	0.000	3.583
P8->OUT2	AVG	3.583	0	0.000	3.583
P8->OUT2	DAVG	3.583	0	0.000	3.583

表 6: 代表性 OD 对的路径指标对比

表 6 与图 3 相互印证: A* 的优势在于稳健可达和离散化处理, VG 的优势在于几何距离, AVG/DAVG 的优势在于用附加代价表达转向和风险偏好。对于上层调度而言, 这些算法没有绝对优劣, 关键是它们提供了不同路径风格。

4.9 mixed 多候选路径库

单一 P1 算法对同一 OD 通常只输出一条路径, 无法充分体现“路径可选择性”。因此本文通过 `build_mixed_paths.py` 融合 `basic_astar`、`vg`、`avg` 和 `davg` 的输出, 生成 `data/processed/p1/mixed/`。mixed 路径库会给 `path_uid` 加算法前缀, 并在几何去重后保留多条候选。本算例融合后平均每个 OD 有 1.77 条候选路径, 最多 3 条。

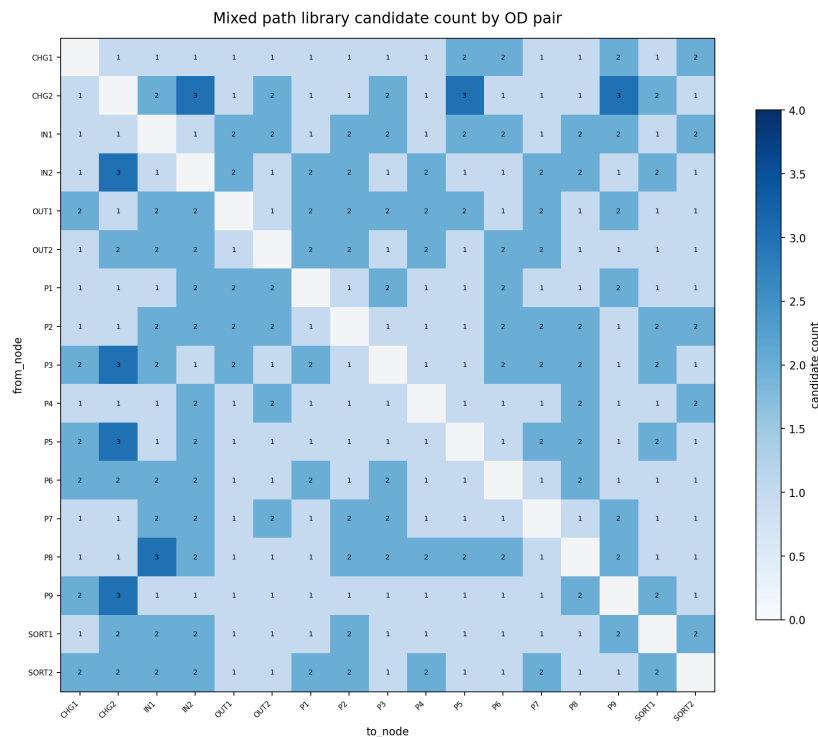


图 4: mixed 路径库中各 OD 对的候选路径数量

图 4 展示 mixed 路径库在 OD 层面的候选数量分布。统计结果显示, 152 个 OD 对只有 1 条几何唯一候选, 112 个 OD 对有 2 条候选, 8 个 OD 对有 3 条候选。候选数量不是越多越好: 若多个算法输出相同或近似相同的几何路径, 去重后只保留一条即可; 若算法偏好导致路径明显不同, 则保留多条供 P2 选择。

mixed 的意义在于把路径规划层的多种偏好上交给调度层。P2 可以在任务排序、时间窗、电量和目标规划约束下决定使用哪条路径, 而不是被某一个底层路径算法固定住。例如, 在静态场景中, VG 可能提供最短通行时间; 在强调执行平滑性时, AVG 路径可能更合适; 在动态封锁或风险区域出现后, DAVG 路径可能为 P4 提供绕行选择。这样, P1 与 P2/P4 的关系从“单路径输入”变成“候选路径集合输入”。

4.10 输出接口

P1 的主要输出位于 `data/processed/p1/<algorithm>/`:

- `key_nodes.csv`: 参与路径规划的关键节点;
- `path_cost.csv`: 面向 P2 的 OD 路径成本表;
- `path_grid_cells.csv`: 路径穿越栅格或可视图顶点记录;
- `path_trajectory_samples.csv`: 面向 P3/P4 的相对时间轨迹采样;
- `path_cost_matrix_time.csv` 和 `path_cost_matrix_total.csv`: 时间矩阵和综合成本矩阵。

其中, `path_cost.csv` 提供 `from_node`、`to_node`、`path_uid`、`distance`、`travel_time`、`turn_cost`、`dynamic_cost`、`total_cost` 和 `planning_status`。P2 加载时会过滤 `planning_status != ok` 的路径, 避免不可达 OD 被误用于调度。

`path_trajectory_samples.csv` 提供相对出发时刻的二维轨迹样本, 包括 `offset_time`、`x`、`y`、`theta`、`speed` 和 `footprint_radius`。P3/P4 读取该表后, 将相对时间平移到具体任务的绝对开始时间, 从而形成可检测冲突的时空轨迹。

通过上述表结构, P1 完成了从“二维空间路径”到“调度成本矩阵”和“时空轨迹样本”的转换。路径几何用于解释算法差异, 时间矩阵用于 P2 的任务排序与衔接, 轨迹采样则用于 P3/P4 的冲突检测与动态重排。

4.11 与后续模块的关系

P1 的设计边界是“生成候选路径与轨迹样本”, 不在 P1 内部求解全局多机器人调度。这样可以让路径算法与调度算法解耦: 路径规划层只关心二维可达性、距离、转弯和动态风险; P2-P4 再根据任务、时间窗、电量和冲突约束选择和调整这些候选路径。

这种分层也解释了为什么 P1 需要保留多种路径算法。若 P1 只输出一张最短路径矩阵, P2 的路径选择变量就会退化; 若 P1 输出多候选路径, P2 才能在“短距离、少转弯、

低风险、低冲突潜力”之间进行组合优化。P3/P4 则进一步使用轨迹采样表验证这些任务级路径是否在时间维度上真正可执行。

5 P2：任务分配与任务排序模型

5.1 输入、输出与决策内容

P2 负责决定三个问题：

1. 每个任务由哪台 AMR 执行；
2. 每台 AMR 内部按什么顺序执行任务；
3. 每段空驶和载货移动选用哪条 P1 候选路径。

P2 输入为 `tasks.csv`、`amrs.csv` 和 P1 输出目录中的 `path_cost.csv`，输出为：

- `data/processed/p2/assignment_result.csv`：任务级调度表；
- `data/processed/p2/amr_sequence_summary.csv`：AMR 任务链摘要。

输出表中保留了 P3 所需的接口字段，例如 `transition_path_uid`、`loaded_path_uid`、`estimated_start_time`、`estimated_finish_time`、`estimated_waiting`、`energy_used` 和 `battery_after`。

5.2 P2 数学模型

设 K 为 AMR 集合， J 为任务集合， P_i 和 D_i 分别为任务 i 的取货点和送货点。P2 的核心是一个带任务指派、任务排序和路径选择的 0-1 混合整数规划。主要变量如下：

变量	含义
$x_{ki} \in \{0, 1\}$	任务 i 是否分配给 AMR k
$s_{ki} \in \{0, 1\}$	任务 i 是否为 AMR k 的首个任务
$u_{kij} \in \{0, 1\}$	AMR k 是否在任务 i 后紧接执行任务 j
$S_i, C_i \geq 0$	任务 i 的开始时间与完成时间
$T_i \geq 0, \text{late}_i \in \{0, 1\}$	任务 i 的延期量与是否延期
B_i	完成任务 i 后的剩余电量
$C_{\max}, L_k, L_{\max}, L_{\min}$	系统完工时间、AMR 时间负载及其最大/最小值

表 7: P2 主要决策变量

任务唯一分配约束为：

$$\sum_{k \in K} x_{ki} = 1, \quad \forall i \in J.$$

AMR 任务序列约束使用虚拟起点 0_k 和虚拟终点 $\bar{0}_k$ 表示:

$$\sum_{j \in J} y_{k,0_k,j} \leq 1, \quad \sum_{i \in J} y_{k,i,\bar{0}_k} \leq 1, \quad \forall k \in K.$$

$$\sum_{j \in J \cup \{\bar{0}_k\}} y_{kij} = x_{ki}, \quad \sum_{i \in J \cup \{0_k\}} y_{kij} = x_{kj}.$$

若显式使用首任务变量 s_{ki} , 也可写成:

$$s_{kj} + \sum_{i \neq j} u_{kij} = x_{kj}, \quad \sum_{j \in J} s_{kj} \leq 1.$$

该约束保证每台 AMR 的任务形成一条线性链, 避免同一任务出现多个前驱、多个后继或游离于 AMR 起点之外的子环。

时间递推约束为:

$$S_i \geq e_i, \quad C_i \geq S_i + s_i + \tau_{P_i D_i}^q - M(1 - x_{ki}),$$

$$S_j \geq C_i + \tau_{D_i P_j}^q - M(1 - y_{kij}).$$

其中 τ^q 是按照 AMR 速度系数和载货速度系数修正后的实际通行时间。本文取载货速度系数为 0.90, 空驶和载货移动分别计算时间、成本和能耗。

具体而言, 若 P1 给出的基准通行时间为 τ_{ab} , AMR k 的速度系数为 v_k , 则空驶和载货时间分别为:

$$t_{k,a \rightarrow b}^{empty} = \frac{\tau_{ab}}{v_k}, \quad t_{k,i}^{loaded} = \frac{\tau_{P_i D_i}}{0.9v_k}.$$

延期变量定义为:

$$T_i \geq C_i - l_i, \quad T_i \geq 0.$$

电量安全约束通过任务链仿真检查实现。空驶、载货和服务分别按距离或服务时间计耗电; 若某一任务插入导致 AMR 剩余电量低于安全阈值, 则评价指标中的 `energy_violation_count` 增加, 字典序目标会优先压低该违反数。

5.3 目标规划口径

P2 采用目标规划中的优先级因子思想, 将评价函数写成字典序目标:

$$\min \text{lex} \left(F_0^{\text{path}}, F_0^{\text{battery}}, F_1^{\text{priority}}, F_1^{\text{late}}, F_2, F_3 \right).$$

其中:

$$F_1^{\text{priority}} = \sum_i \text{priority}_i \cdot \text{late}_i, \quad F_1^{\text{late}} = \sum_i \text{late}_i,$$

$$F_2 = 30 \sum_i \text{priority}_i T_i + 20 \sum_i T_i + 5C_{\max},$$

$$F_3 = 2 \cdot \text{empty_cost} + 10(L_{\max} - L_{\min}) + 1 \cdot \text{total_energy}.$$

字典序目标的含义是：先保证路径存在和电量安全，再减少高优先级延期任务和延期任务数量，最后优化时间效率与运行成本。这对应目标规划中的 $P_1 \gg P_2 \gg P_3$ 结构：高优先级任务延期属于服务承诺，不能被任何路径成本或负载均衡收益抵消。求解时可采用序贯解法，即先最小化高优先级目标，将其最优值固定后再进入下一层目标。

5.4 求解方法实现

P2 比较四类方法：

方法	实现内容	对应知识点
m0	修复版贪心，按链式位置插入并检查路径、电量和时间	对照基线
m1	两阶段法：阶段一 0-1 指派模型，阶段二车内 Held-Karp 动态规划排序	整数规划、分支定界、动态规划
m2	联合 MILP + 五层目标规划序贯求解，使用分支定界/割平面求解	整数规划、目标规划、分支切割
m3	regret-2 插入构造 + relocate/swap/reroute 局部搜索	局部最优思想、启发式优化

表 8: P2 求解方法

本文主实验采用 `m2_milp_goal_programming+mixed`。在 `amr_sequence_summary.csv` 中，4 台 AMR 的任务链为：

AMR	任务序列	完成时间	剩余电量
AMR1	T06 → T03 → T07	40.293444	54.7776
AMR2	T02 → T05 → T10	40.840111	48.0854
AMR3	T01 → T09 → T12	38.944691	67.5520
AMR4	T04 → T08 → T11	38.596970	43.4962

表 9: P2 当前静态任务分配结果

该 P2 结果满足缺失衔接路径数 0、缺失载货路径数 0、电量阈值违反数 0、静态任务延期数 0。

6 P3: 时空轨迹展开与冲突修复

6.1 模块定位与输入输出

P2 输出的是任务级调度表, 已经决定了每个任务由哪台 AMR 执行、每台 AMR 内部按什么顺序执行任务, 以及每段空驶和载货移动选用哪条 P1 候选路径。但任务级计划并不直接等价于可执行轨迹。两台 AMR 即使在任务时间窗、电量和路径可达性上都满足要求, 仍可能在同一时间进入同一狭窄通道、同一取货点或同一分拣点, 导致二维 footprint 冲突。因此, P3 的作用是把 P2 的静态任务计划推进到执行层, 生成带绝对时间的二维时空轨迹, 并在轨迹层面检测和修复多 AMR 冲突。

从模块边界看, P3 不重做 P2 的全局任务分配, 也不临时生成新的几何路径。它在 P2 给定的任务分配和任务顺序基础上进行局部执行修复; 若需要换路, 也只从 P1 mixed 路径库中选择同 OD 的候选路径; 若需要换序, 也只允许同一 AMR 内部相邻任务的局部交换。这样的边界保证了 P2、P3 和 P4 的职责清晰: P2 决定“谁执行、按什么顺序执行”, P3 判断“这些任务在二维时空中能否安全执行”, P4 则在 P3 无冲突基准计划上处理动态封锁、AMR 延误和新增任务。

从系统分工看, P3 的价值不在于替换 P1 的路径生成或 P2 的任务级优化, 而在于把前序模块的静态计划补上执行层闭环。P1 提供可走路径, P2 形成较优任务表, P3 进一步给出可检查、可修复、可交付给 P4 的时空轨迹。因此, P3 的工作量主要体现在轨迹展开、冲突检测、局部修复、过程日志、可视化和下游接口的一体化上; 其结果也更容易通过“初始冲突数、修复后冲突数、轨迹采样表、动画快照”形成清晰证据链。

P3 的核心输入为:

- `data/processed/p2/assignment_result.csv`: P2 输出的任务分配、任务顺序、时间窗、路径选择和电量结果;
- `data/processed/p1/mixed/path_cost.csv`: P1 mixed 路径库中的路径成本和同 OD 候选, 用于换路候选生成;

- `data/processed/p1/mixed/path_trajectory_samples.csv`: P1 路径几何采样点，用于展开二维时空轨迹。

P3 的输出为:

- `schedule_result.csv`: 修复后的任务级时间表，表头严格沿用 P2 输出格式;
- `trajectory_schedule.csv`: 修复后的 0.1 秒粒度时空轨迹表，是 P4 做动态事件判断和二次冲突检查的主要轨迹接口;
- `conflict_log.csv`: 每轮冲突修复的过程日志;
- `repair_summary.csv`: 三种修复模式的汇总指标和最终选择。

其中，`schedule_result.csv` 保持 P2 表头兼容，是为了降低 P4 接入成本; P4 可以继续按 P2 的字段结构读取任务级时间表，只需要把 P3 更新后的 `estimated_start_time`、`estimated_finish_time`、`estimated_delay` 和 `estimated_waiting` 作为最终执行时间读取。另一方面，`trajectory_schedule.csv` 记录最终实际使用的轨迹点和等待片段，是 P3 相比 P2 新增的执行层信息。

6.2 轨迹展开模型

P3 首先按 `amr_id` 和 `sequence_order` 遍历 P2 任务表。对每台 AMR，算法从初始节点出发，依次处理该 AMR 的任务链。对每个任务 i ，执行过程被拆分为四类基本片段:

1. `transition`: 从当前节点空驶到任务取货点;
2. `wait_at_pickup`: 若早于任务最早开始时间到达，则在取货点等待;
3. `service_at_pickup`: 取货服务时间;
4. `loaded`: 从取货点载货运行到送货点。

若 P3 插入修复等待，还会产生若干静止轨迹片段。例如，`repair_wait_at_node` 表示任务开始前在节点等待，`mid_path_wait_transition` 表示空驶路径中途暂停，`mid_path_wait_loaded` 表示载货路径中途暂停。所有片段最终都被转化为统一的轨迹样本:

$$(k, i, o, s, t, x(t), y(t), \theta(t), v(t), \rho),$$

其中 k 为 AMR， i 为任务， o 为车内执行序号， s 为轨迹段类型， t 为全局绝对时间， $(x(t), y(t))$ 为二维坐标， $\theta(t)$ 为朝向， $v(t)$ 为速度， ρ 为 AMR footprint 半径。对移

动片段，P3 按 P2 实际行驶时间对 P1 的路径采样点做时间缩放和插值；对等待和服务片段，P3 在固定坐标处生成速度为 0 的轨迹点。

这一展开过程有两个作用。第一，它把任务级的开始时间和完成时间转化为实际的空间占用过程，使后续冲突检测可以基于二维坐标进行。第二，它显式保留等待和服务过程，避免只检测移动轨迹而漏掉取货点、分拣点等关键节点的静止占用。对于多 AMR 仓储调度，这一点很重要，因为冲突不只发生在移动通道中，也可能发生在任务服务点。

6.3 冲突检测模型

P3 检测两类冲突。第一类是空间 footprint 冲突。设 $X_k(t) = (x_k(t), y_k(t))$ 表示 AMR k 在时刻 t 的二维位置， ρ_k 为 footprint 半径， σ 为安全裕量。当两台 AMR 在相近时间点上的中心距离小于半径与安全裕量之和时，判定为空间冲突：

$$\|X_{k_1}(t) - X_{k_2}(t)\| < \rho_{k_1} + \rho_{k_2} + \sigma.$$

当前实验参数为：

$$\rho = 0.25, \quad \sigma = 0.10, \quad \text{TIME_TOLERANCE} = 0.25, \quad \Delta t = 0.10.$$

因此，两台 AMR 的空间冲突距离阈值为 $0.25 + 0.25 + 0.10 = 0.60$ 。其中 TIME_TOLERANCE 用于比较相近时间点上的两台 AMR 位置， Δt 是轨迹展开的采样间隔。该检测方式直接对应题目中的二维 footprint 冲突要求，比只在抽象图边上设置容量更贴近仓库平面场景。

第二类是节点容量冲突。对 P^* 、 SORT^* 、 OUT^* 等关键节点，同一时刻最多允许一台 AMR 占用或作业：

$$\sum_{k \in K} I(k \text{ occupies node } v \text{ at } t) \leq 1.$$

节点容量冲突用于补充空间距离判断。即使两台 AMR 的中心距离没有触发普通空间冲突，只要它们在同一时刻静止占用同一取货点、分拣点或出库点，也应视为不可执行方案。P3 将等待、服务和中途停车都纳入轨迹表，因此节点容量约束可以覆盖移动之外的静止作业过程。

需要说明的是，P3 的冲突检测基于离散采样。采样方法直观、可复现，也便于与 P4 动态事件检查共享同一轨迹表；但它的精度依赖采样间隔。如果采样间隔过大，可能漏掉持续时间很短的冲突；如果采样间隔过小，则会显著增加轨迹行数和两两比较开销。当前 0.1 秒粒度在本算例中能够满足展示和验证需要，后续在高速或高密度场景中仍需进一步分析采样误差。

6.4 局部修复策略

P3 采用启发式局部修复方法。每一轮修复首先选择当前最早发生的冲突，然后根据冲突涉及的 AMR、任务和路径生成候选动作。候选动作包括：

- `wait_before_task`: 在某个任务开始前增加等待；
- `mid_path_wait`: 在冲突点前的路径中途停车等待；
- `reroute`: 从 P1 mixed 路径库中选择同 OD 替代路径；
- `resequence`: 对同一 AMR 内部相邻任务做局部交换。

每个候选动作都会形成一个新的修复状态。P3 根据该状态重新生成任务时间表和轨迹表，再重新检测冲突并计算目标函数。若某个候选动作能够改善“剩余冲突数、目标函数值”这一二元指标，则采用其中最优动作进入下一轮；若没有任何候选动作能够改善当前状态，则停止该模式的修复。

当前 P3 比较三种修复模式：

1. `wait`: 只通过等待修复冲突；
2. `wait_reroute`: 在等待基础上允许同 OD 换路；
3. `wait_reroute_resequence`: 在等待和换路基础上允许同车相邻换序。

其中，`mid_path_wait` 是当前算例中最主要的实际修复动作。它不是把整项任务简单后移，而是在 AMR 沿原路径行驶到冲突点前某个位置时停车等待，待对方通过后继续沿原路径前进。当前提前停车候选为 0.8、1.0、1.5、2.0、3.0、4.0 秒。与任务开始前等待相比，中途等待更接近实际执行层让行，因为它只在冲突发生前局部插入等待，而不是把整个任务链从一开始整体后移。

在得到 0 冲突方案后，P3 还会执行等待压缩。具体做法是尝试减少已经插入的等待时间，每次缩短后重新检测冲突；只有仍然保持 0 冲突的缩短方案才会被接受。这一设计避免了“为了消除冲突而无限增加等待”的问题，使最终方案保留满足无冲突要求所需的较小等待量。

6.5 目标函数与选择规则

P3 的选择规则是先最小化剩余冲突数，再最小化执行层目标函数：

$$J = 100 \cdot total_delay + 5C_{\max} + wait_added \\ + 2 \cdot reroute_count + 20 \cdot resequence_count + 20 \cdot initial_wait_added.$$

该目标函数服务于执行层冲突修复，而非重新求解 P2 的全局任务分配问题。其含义可以分为三层。第一，剩余冲突数具有最高优先级，不可执行方案即使完工时间较短也不应被选中。第二，在冲突数相同的情况下，模型尽量减少任务延期和 $C_{\max}$ ，避免修复动作破坏原计划的服务质量。第三，目标函数对等待、换路、换序和开局等待设置惩罚，避免为了局部冲突修复而过度扰动原计划。

三种修复模式的当前结果如下：

模式	初始冲突	剩余冲突	总延期	$C_{\max}$	新增等待	是否选中
wait	21	0	0.0	46.59697	16.0	否
wait_reroute	21	0	0.0	46.59697	16.0	否
wait_reroute_resequence	21	0	0.0	46.59697	16.0	是

表 10: P3 冲突修复模式对比

表 10 显示，三种模式都能将 21 个初始冲突修复为 0，且总延期、 $C_{\max}$ 和新增等待完全相同。最终被标记为 `wait_reroute_resequence`，是因为在并列情况下该模式作为能力最完整的方案被选中。需要强调的是，本算例中实际换路次数和换序次数均为 0，因此当前结果只能说明等待，尤其是载货路径中途等待，是本组数据下的主要修复动作，换路和换序在当前地图下未带来超过停车等待的明显收益。

但从机制上看，后两类动作仍然有保留价值。若仓库地图存在差异明显的同 OD 候选路径，且冲突集中在某些狭窄通道或瓶颈节点，提高等待惩罚或降低换路惩罚后，换路就可能通过绕开拥堵区减少等待或降低 $C_{\max}$ 。若同一 AMR 的相邻任务在时间窗上具有一定弹性，且当前任务链会把车辆推到同一瓶颈时刻，则同车局部换序也可能通过改变到达时序减少冲突。换言之，等待是当前算例的实际发力点，换路和换序更像为高拥堵、多路径差异或强时序耦合场景预留的高级候选动作。

6.6 可视化结果与轨迹解释

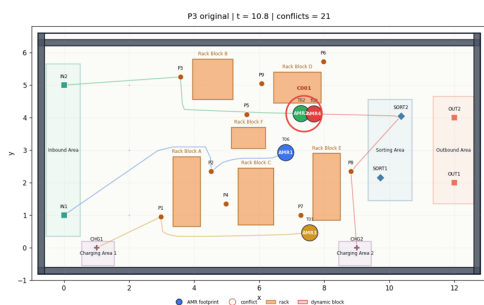
为了更直观地说明 P3 的作用，本文从 `outputs/p3` 中的动画结果截取了原始轨迹和最终修复轨迹在同一时刻的画面，如图 5 所示。左图对应 P2 原始计划直接展开后的运行状态，右图对应 P3 选中方案 `wait_reroute_resequence` 修复后的运行状态。两张图取自同一近似时刻，便于比较 P3 修复前后的空间占用差异。

图 5 左侧可以看到，在原始 P2 轨迹中，AMR2 与 AMR4 在货架区中上部附近距离过近，触发 C001 空间冲突。该冲突不是任务表层面能够直接发现的问题，因为 P2 只知道两台 AMR 的任务顺序、开始时间和路径编号，并没有逐时刻比较二维 footprint。右侧修复后，同一时刻 AMR2 与 AMR4 的空间位置已经拉开，图中不再标记冲突。这说明 P3 的等待修复并不是简单修改一个汇总指标，而是改变了 AMR 在具体时刻、具体位置上的空间占用。

从轨迹形态看，P3 修复后并没有让 AMR 离开原路径绕行，也没有跨 AMR 重分配任务，而是通过路径中途停车让行改变相对到达时刻。当前最终轨迹表中出现 163 条 `mid_path_wait_loaded` 记录，正是这种中途等待的轨迹化表达。换句话说，P3 把“让一

Original P2 trajectory

frame 43/164, t approx 10.75s



P3 repaired trajectory

frame 43/187, t approx 10.75s

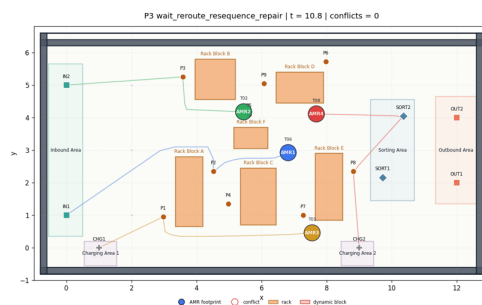


图 5: P3 原始轨迹与修复后轨迹快照对比

台车等一下”落实为可检查的时空点：等待发生在哪条路径上、等待时刻是多少、等待位置坐标是多少、等待期间 footprint 如何占用空间，都被记录在 `trajectory_schedule.csv` 中。

这一可视化也解释了为什么 P3 输出需要同时保留 `schedule_result.csv` 和 `trajectory_schedule.csv`。前者能够告诉下游模块任务完成时间、等待时间和延期情况，后者则能告诉下游模块每台 AMR 在任意时刻实际位于仓库平面的哪个位置。对于 P4 来说，动态封锁区、AMR 延误窗口和新增任务插入都需要基于后一类轨迹信息判断，单纯读取任务级时间表是不够的。

除图 5 所示的静态快照外，P3 还在 `outputs/p3` 中输出了原始轨迹动画和三种修复模式动画。动画中每台 AMR 用带半径的圆表示 footprint，尾迹表示已经走过的轨迹，红色圆圈标识冲突位置；当某台 AMR 完成全部任务后，其显示会淡化，表示不再参与后续冲突检测。需要注意的是，动画只承担可视化作用，不能替代 CSV 文件中的轨迹与指标记录；真正用于判断最终方案是否无冲突的依据仍是 `repair_summary.csv` 和 `trajectory_schedule.csv`。

这些动画的意义不是替代 CSV 结果，而是用于解释 CSV 结果背后的执行过程。表格能够说明“冲突数从 21 变成 0”，动画则能够说明“冲突发生在哪里、哪台车等待、等待后空间关系如何变化”。将静态快照、修复摘要表和冲突日志结合起来，可以形成较完整的证据链：原始任务级计划存在执行冲突，P3 通过局部中途等待改变轨迹时序，最终输出的时空轨迹重新检测后无冲突。

6.7 输出文件与复现实验检查

P3 在输出目录中保留了四类结构化文件。它们分别服务于最终执行计划、轨迹级验证、修复过程追踪和模式对比，如表 11 所示。

输出文件	作用
<code>schedule_result.csv</code>	修复后的任务级时间表，表头沿用 P2 输出格式，记录每个任务的最终开始时间、完成时间、等待时间、延期、路径编号和电量变化。
<code>trajectory_schedule.csv</code>	修复后的二维时空轨迹表，当前共有 1800 条采样记录，是判断动态封锁、AMR 延误和二次冲突的主要执行层输入。
<code>conflict_log.csv</code>	修复过程日志，记录每轮冲突时间、冲突类型、候选动作、目标 AMR、目标任务、等待量和修复后冲突数。
<code>repair_summary.csv</code>	三种修复模式的指标汇总，记录初始冲突、剩余冲突、总延期、 $C_{\max}$ 、新增等待、换路次数、换序次数和最终选中标记。

表 11: P3 结构化输出文件

从复现实验角度看，P3 的结果可以分为两层验证。第一层是指标验证：读取 `repair_summary.csv`，检查最终选中模式的 `selected=1` 且 `remaining_conflicts=0`。当前结果中，三种模式的初始冲突数均为 21，剩余冲突数均为 0，最终选中 `wait_reroute_resequence`。第二层是轨迹验证：读取 `trajectory_schedule.csv`，重新基于绝对时间、二维坐标和 footprint 半径检测空间冲突，并基于轨迹段类型检测节点容量冲突。只有这两层同时成立，才能说明 P3 的输出已经从任务级可行推进到轨迹级可执行。

进一步观察轨迹段分布，可以看到最终轨迹并不是单纯的移动轨迹集合，而是包含了移动、服务、等待和让行过程。当前 1800 条采样记录中，`loaded` 为 796 条，`transition` 为 537 条，`service_at_pickup` 为 282 条，`wait_at_pickup` 为 22 条，`mid_path_wait_loaded` 为 163 条。图 6 将修复日志和轨迹段分布放在同一张图中：左侧显示选中模式下剩余冲突数按 $21 \rightarrow 16 \rightarrow 7 \rightarrow 4 \rightarrow 2 \rightarrow 0$ 逐步下降，右侧显示最终轨迹中载货、空驶、服务和中途等待的记录规模。

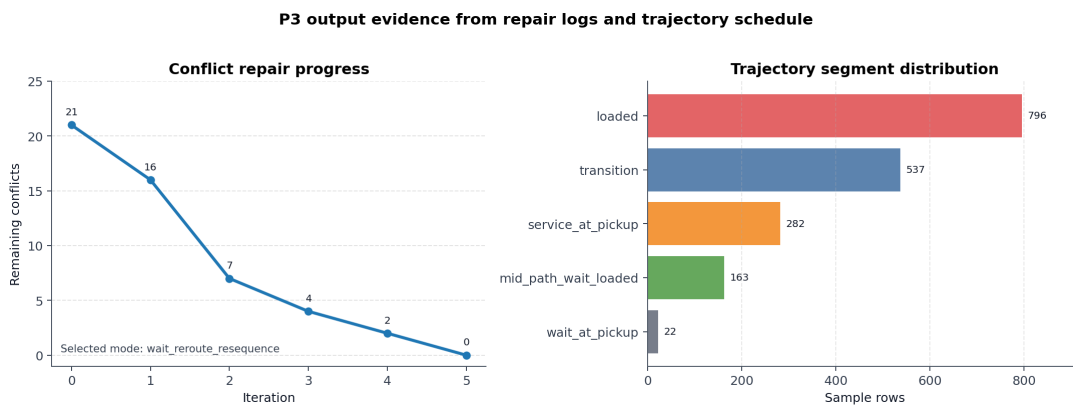


图 6: P3 修复日志与轨迹段分布

图 6 进一步说明, P3 的主要修复动作落在载货路径中途等待上, 而不是在任务开始前整体延后; 这也与 `reroute_count=0`、`resequence_count=0` 的统计结果一致。

在当前算例中, 等待、换路和同车相邻换序三类动作都被纳入候选框架, 但真正被最终轨迹采用的是中途等待让行。换路和换序机制的工程意义在于为更复杂的冲突场景预留扩展能力, 而不是在本次实验中贡献了额外数值收益。这样的表述能够避免把当前结果过度解释为全局最优或高级动作显著改进, 同时也能体现 P3 在接口、日志和可复现性上的完整性。

6.8 P4 接口与工程可解释性

P3 与 P4 的衔接主要依赖两个文件。第一, `schedule_result.csv` 保持 P2 表头, 更新了修复后的任务时间、等待、路径、电量和方法标记。P4 若只需要任务级基准计划, 可以直接读取该文件, 并将其中的 `estimated_*` 字段映射为内部使用的开始时间、完成时间、延期和等待时间。第二, `trajectory_schedule.csv` 保存最终实际执行轨迹, 包括 `transition`、`loaded`、`service_at_pickup`、`wait_at_pickup` 和 `mid_path_wait_loaded` 等片段。P4 若要判断动态封锁区、AMR 延误窗口或剩余轨迹冲突, 应优先使用该轨迹表。

这一接口设计有两个优点。首先, P3 没有破坏 P2 的任务级数据结构, 降低了下游模块读取成本。其次, P3 额外输出了最终轨迹, P4 不需要重新展开 P1 的全部候选路径, 也不需要直接使用 P2 的原始计划。特别是对于中途等待片段, P4 可以把它视为普通静止轨迹点: 它有绝对时间、坐标、footprint 半径和速度 0, 因此可以直接参与动态区域封锁和二次冲突判断。

P3 的修复过程也具有可解释性。`conflict_log.csv` 记录每轮修复的模式、迭代次数、冲突时间、冲突类型、修复动作、目标 AMR、目标任务、等待时间和修复后冲突数。`repair_summary.csv` 则记录三种模式的初始冲突、剩余冲突、延期、 $C_{\max}$ 、等待、换路、换序和目标函数值。

6.9 方法局限与全局最优讨论

P3 当前更适合作为启发式执行层修复模块, 而不是全局最优协调模块。其主要优点是结构清晰、动作直观、输出稳定, 并且能够与 P4 顺利衔接; 主要不足是不能保证全局最优, 且修复结果可能受到冲突处理顺序和候选动作集合的影响。当前实验中, 换路和同车相邻换序虽然已经纳入代码框架, 但最终没有实际被采用, 因此高级修复动作仍需要在更高冲突密度的算例中进一步验证。

理论上, 可以将 P3 建模为全局最优问题, 例如使用 MILP、CP-SAT、CBS/MAPF 或时空网络流模型。但在本项目中直接采用完整全局最优算法会带来明显困难。第一, 时间离散粒度为 0.1 秒, 当前 $C_{\max}$ 约为 46.6 秒, 意味着单个算例就接近 466 个时间步。若在每个时间步描述每台 AMR 的位置、路径段、等待状态和任务状态, 变量规模会随 AMR 数量、任务数量和候选路径数快速增长。第二, 二维 footprint 冲突本质上是几何

距离约束，若用 MILP 精确表达，需要处理平方距离、二阶锥约束或大量 Big-M 互斥约束；若用网格近似，又会在几何精度和模型规模之间产生取舍。

第三，P3 的动作之间强耦合。对某台 AMR 加 2 秒等待，可能消除当前冲突，却引入后续任务的新冲突；换路可能缩短空间冲突，却改变到达时间；换序可能改善局部等待，却改变 P2 给出的车内任务链。因此，全局模型需要同时考虑等待位置、等待时长、路径选择、任务顺序和所有 AMR 的相互影响，求解时间和模型解释都会变得复杂。第四，如果 P3 允许跨 AMR 重分配任务或大范围换序，它会与 P2 的职责重叠；如果动态事件后重新求全局模型，又会与 P4 的滚动重排职责重叠。

因此，本项目 P3 并不是严格的全局最优求解模块，而是项目分工时确定的执行层可行性修复模块。更合理的后续改进方向是在当前启发式框架上加入局部优化：例如只对冲突发生前后的有限时间窗建模，只对冲突图中的相关 AMR 组成小规模冲突组，或先由启发式生成有限个等待、换路和换序候选，再用整数规划从候选动作中选择组合。这样既能提高局部修复质量，又能保留当前 P3 的可解释性和接口稳定性。

7 P4：动态事件与滚动时域重排

7.1 模块定位与输入输出

P4 位于 P3 之后，是本项目从“静态可执行计划”走向“动态仓储运行”的关键模块。P3 已经把 P2 的任务级计划展开为二维时空轨迹，并通过等待修复将初始轨迹冲突数降为 0；实际仓储系统在执行过程中还会遇到局部通道封锁、某台 AMR 暂时不可用、临时新增任务等扰动。P4 在 P3 给出的无冲突基准轨迹上进行滚动时域局部重优化：保留已经执行的计划，把受动态事件影响的未来任务释放到重排池中，并重新确定 AMR 序列、开始时间和必要等待。

P4 的输入包括：

- `data/processed/p3/schedule_result.csv`: P3 修复后的任务级基准计划；
- `data/processed/p3/trajectory_schedule.csv`: P3 修复后的二维时空轨迹；
- `data/raw/dynamic_events.csv`: 动态事件表，包含封锁、延误和新增任务；
- `data/raw/tasks.csv`、`data/raw/amrs.csv`: 静态任务与 AMR 初始参数；
- `data/processed/p1/mixed/path_cost.csv` 与 `path_trajectory_samples.csv`: 候选路径成本与轨迹采样。

P4 的输出为：

- `reschedule_result.csv`: 任务重排结果，记录原 AMR、新 AMR、原开始/完成时间、新开始/完成时间、变更原因和路径编号；

- `dynamic_event_impact.csv`: 每个动态事件的影响范围, 包括受影响任务和 AMR;
- `reschedule_summary.csv`: 动态重排后的可行性与目标函数指标;
- `trajectory_schedule.csv`: P4 最终二维时空轨迹;
- `p4_method_comparison.csv` 与 `p4_weight_sensitivity.csv`: 方法对比和目标口径记录;
- `outputs/p4` 下的甘特图、轨迹地图、指标图、方法对比图和动态 GIF。

从系统分工看, P4 的核心价值在于保持计划稳定性的同时恢复动态可行性。它读取 P3 已经消除冲突的基准计划, 在事件到达时冻结历史任务, 只对尚未固定的未来任务池进行局部插入、局部等待和冲突修复。这样既能保留 P2/P3 已经形成的高质量计划, 又能对实际运行中的突发事件做出响应。

7.2 图形化运行过程

为了避免 P4 只停留在公式和表格层面, 本节先给出本次动态重排的直接可视化结果。图 7 从时间维度展示了 P3 基准计划与 P4 重排计划的差异: P4 没有全局推翻原计划, 而是在动态事件之后对局部未来任务进行调整。新增任务 T13 被插入 AMR4 的未来序列, AMR 延误事件导致相关任务在可用时间之后重新衔接, 最终所有任务仍形成可执行的时间链。

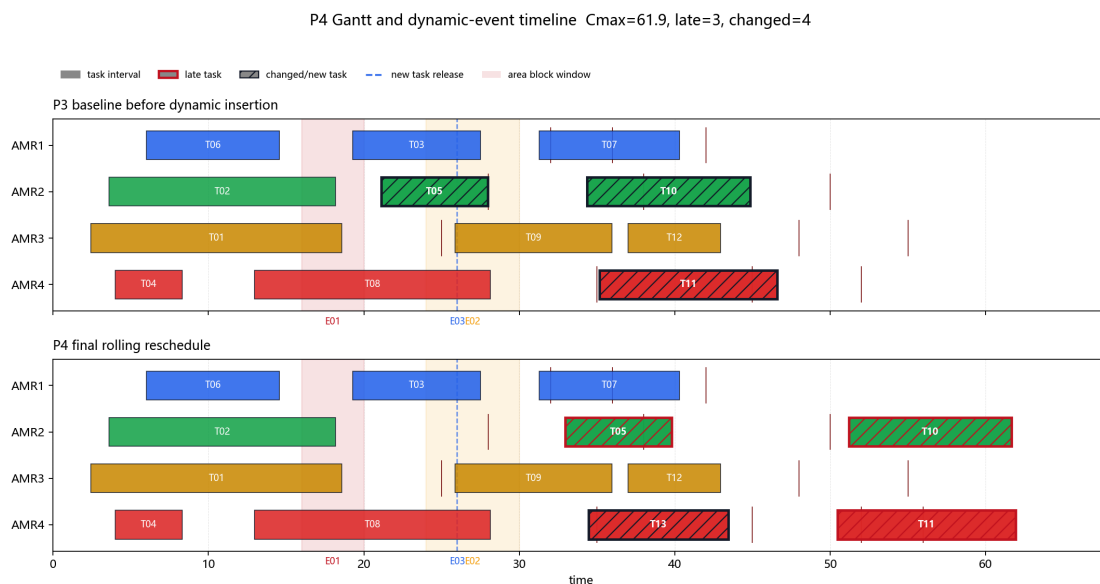


图 7: P4 动态事件后的任务时间轴与重排结果

图 8 从空间维度展示最终二维轨迹。该图的作用不是重复 P1 的路径搜索, 而是验证 P4 输出的任务序列在仓库平面上可以转化为连续轨迹, 并且能够与 P3 的冲突检测

接口衔接。换言之，P4 的解不是单纯的任务表更新，而是已经落到二维地图、候选路径和轨迹采样层面的动态调度方案。

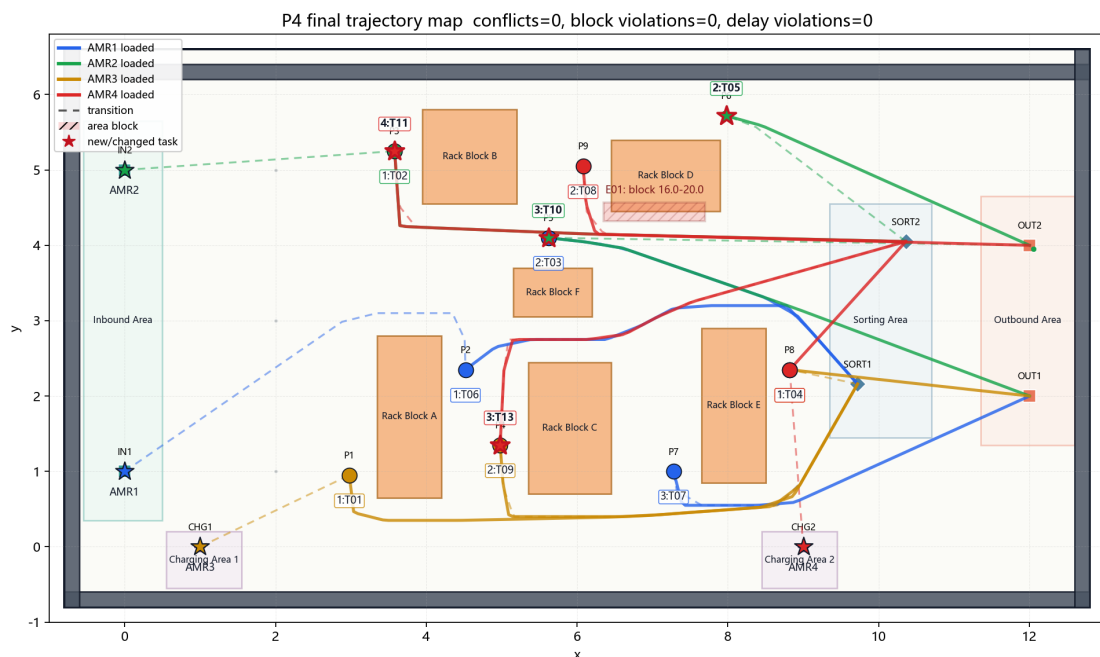


图 8: P4 重排后的二维轨迹分布

图 9 进一步从动态过程角度展示重排逻辑。四个快照分别对应 P3 基准计划、动态事件触发、新任务插入与局部修复、最终可执行方案。完整 GIF 已保存在 `outputs/p4/p4_dynamic_reschedule.gif`；报告正文采用抽帧图，是为了保证 PDF 编译后仍能静态呈现动态过程。



图 9: P4 动态重排过程快照

因此，P4 的“调整”具体体现在三个层级：在任务层，受动态事件影响的任务被释放并重新插入；在时间层，AMR 的可用时间和任务开始时间被重新递推；在空间层，最终方案重新生成二维轨迹并交给 P3 冲突检测逻辑验证。后文的数学模型用于解释这些图背后的决策变量、约束条件和目标函数。

7.3 动态事件建模

P4 当前处理三类动态事件，如表 12 所示。三类事件分别对应空间资源变化、车辆可用性变化和任务集合变化。

事件类型	事件参数	处理方式
area_block	封锁区域 (x, y, w, h) 与时间窗 $[\alpha, \beta]$	检查已有轨迹 footprint 是否进入封锁矩形；若影响未来任务，则将相关任务及尾段纳入重排池
amr_delay	目标 AMR k 与不可用时间窗 $[\alpha, \beta]$	识别目标 AMR 上与延误窗口相交或位于其后的任务；释放重叠任务和后续尾段，从延误结束后重新安排
new_task	新任务编号、取货点、送货点、释放时间与优先级	将新任务加入任务集合，在滚动时域内评估不同 AMR 和不同插入位置

表 12: P4 动态事件类型与处理方式

设动态事件集合为 $\mathcal{E}$, 事件 $e \in \mathcal{E}$ 的发生时刻为 t_e 。对于封锁事件, 封锁区域记为矩形 B_m , 时间窗为 $[\alpha_m, \beta_m]$ 。若某条轨迹采样点 $(x_k(t), y_k(t))$ 满足

$$t \in [\alpha_m, \beta_m], \quad (x_k(t), y_k(t)) \in B_m \oplus \rho,$$

其中 ρ 为 AMR footprint 半径, 则该事件与现有轨迹发生空间时间交叠, 需要进入影响分析。当前算例中, E01 的封锁区域在 $[16, 20]$ 内没有与最终轨迹 footprint 相交, 因此影响任务数为 0。

对于 AMR 延误事件, 若 AMR k 的某项任务 i 执行区间 $[S_i, C_i]$ 与延误窗口 $[\alpha, \beta]$ 相交,

$$S_i < \beta, \quad C_i > \alpha,$$

则该任务不能继续被视为已固定计划。P4 会释放该任务及其同车后续尾段, 并把 AMR 的可用时间修正为不早于 β 。这一步是当前实现相对简单后移策略的重要区别: 若一项任务在事件发生前已经开始但在延误窗口内尚未完成, P4 不把它强行保留为历史任务, 而是重新安排其未来可执行部分, 从而避免最终计划仍与延误窗口重叠。

对于新增任务事件, P4 会根据事件释放时间生成动态任务。若事件表未显式给出最晚完成时间, 当前实现采用“释放时刻 +30 秒”作为软期限。该软期限并非硬约束, 而是进入延期指标和目标函数, 用于区分及时插入和过晚插入。

7.4 滚动时域集合划分

设全体任务集合为 J , AMR 集合为 K 。在事件时刻 t_e , P4 将任务划分为固定集合 $J_{fix}(t_e)$ 和可重排集合 $J_{free}(t_e)$:

$$J = J_{fix}(t_e) \cup J_{free}(t_e), \quad J_{fix}(t_e) \cap J_{free}(t_e) = \emptyset.$$

固定集合由两部分构成。第一部分是在 t_e 前已经完成的任务; 第二部分是轨迹已经进入执行状态且没有被当前事件直接破坏的任务。其余任务构成未来任务池。对于 AMR 延误事件, 还需要从固定集合中移除目标 AMR 上与延误窗口重叠的任务:

$$J'_{fix}(t_e) = J_{fix}(t_e) \setminus \{i : k_i = k^{delay}, S_i < \beta, C_i > \alpha\}.$$

这一修正保证延误窗口不会被“已经进入轨迹”这一规则错误保护。

P4 在滚动窗口内保留每台 AMR 的状态。对 AMR k , 状态向量记为

$$\xi_k(t_e) = (a_k, n_k, b_k, r_k),$$

其中 a_k 是最早可用时间, n_k 是当前所在节点, b_k 是剩余电量, r_k 是固定任务数量。若 AMR 受到延误, 则

$$a_k \leftarrow \max(a_k, \beta).$$

后续任务的时间递推从 $\xi_k(t_e)$ 出发，而不是从原始初始点出发。这一点体现了滚动时域方法的本质：过去已经执行的计划成为新的初始条件，未来任务在这个新状态上继续优化。

7.5 混合整数规划抽象

从运筹学角度看，P4 可以写成一个带动态事件约束的动态多 AMR 取送货调度问题，即动态 PDPTW / 多车辆路径调度问题。若把 P4 完整抽象为混合整数规划，可定义如下变量：

变量	含义
$x_{ki} \in \{0, 1\}$	任务 $i \in J_{free}$ 是否分配给 AMR k
$y_{kij} \in \{0, 1\}$	AMR k 是否在任务 i 后紧接执行任务 j
$z_{kijq} \in \{0, 1\}$	AMR k 从任务 i 的送货点到任务 j 的取货点是否选择候选路径 q
$u_{kijq} \in \{0, 1\}$	AMR k 执行任务 i 的载货路径是否选择候选路径 q
$S_i, C_i \geq 0$	任务 i 的开始取货服务时间与完成送货时间
$T_i \geq 0, L_i \in \{0, 1\}$	任务延期量与是否延期
$H_i \in \{0, 1\}, \Delta_i \geq 0$	任务是否相对 P3 基准被扰动，以及开始时间偏移量
$C_{\max} \geq 0$	系统最大完成时间

表 13: P4 动态重排的主要决策变量

任务唯一分配约束为

$$\sum_{k \in K} x_{ki} = 1, \quad \forall i \in J_{free}.$$

对每台 AMR，未来任务必须形成一条从滚动初始状态出发的线性任务链。令 0_k 表示 AMR k 在事件时刻的虚拟起点， $\bar{0}_k$ 表示虚拟终点，则有：

$$\begin{aligned} \sum_{j \in J_{free}} y_{k,0_k,j} &\leq 1, & \sum_{i \in J_{free}} y_{k,i,\bar{0}_k} &\leq 1, \\ \sum_{j \in J_{free} \cup \{\bar{0}_k\}} y_{kij} &= x_{ki}, & \sum_{i \in J_{free} \cup \{0_k\}} y_{kij} &= x_{kj}. \end{aligned}$$

必要时可加入 MTZ 顺序变量 o_{ki} 消除子回路：

$$o_{kj} \geq o_{ki} + 1 - M(1 - y_{kij}), \quad i, j \in J_{free}.$$

路径选择仍严格限制在 P1 mixed 候选路径库中。设 Q_{ab} 为节点 a 到节点 b 的候选

路径集合，则

$$\sum_{q \in Q_{D_i P_j}} z_{kijq} = y_{kij}, \quad \sum_{q \in Q_{P_i D_i}} u_{kiqu} = x_{ki}.$$

如果某一 OD 对不存在可用候选路径，则该方案被记为缺失路径违反，并在字典序目标的硬约束层优先排除。

时间递推约束从滚动初始状态出发。首任务约束为

$$S_j \geq a_k + \sum_{q \in Q_{n_k P_j}} \tau_{kq} z_{k,0_k,j,q} - M(1 - y_{k,0_k,j}),$$

后继任务约束为

$$S_j \geq C_i + \sum_{q \in Q_{D_i P_j}} \tau_{kq} z_{kijq} - M(1 - y_{kij}),$$

任务自身完成时间为

$$C_i \geq S_i + s_i + \sum_{q \in Q_{P_i D_i}} \tau_{kq} u_{kiqu} - M(1 - x_{ki}).$$

延期定义为

$$T_i \geq C_i - l_i, \quad T_i \geq 0, \quad T_i \leq ML_i.$$

AMR 延误窗口可以写成互斥约束。若 AMR k 在 $[\alpha, \beta]$ 内不可用，则其未来任务区间不得与该窗口重叠：

$$C_i \leq \alpha + Mh_{ki} \quad \text{or} \quad S_i \geq \beta - M(1 - h_{ki}),$$

其中 $h_{ki} \in \{0, 1\}$ 。工程求解时，P4 将该互斥关系落实为 AMR 状态更新和最终轨迹验证：受延误 AMR 的可用时间被推迟到延误结束，最终方案再重新统计 `delayed_amr_violation_count`。

封锁区约束与二维轨迹采样相关。设路径 q 在相对时间 r 处的采样位置为 $X_q(r)$ ，若该采样点与封锁区 B_m 相交，则它的绝对通过时刻必须避开封锁时间窗：

$$\text{departure}(q) + r \leq \alpha_m + Mh$$

或

$$\text{departure}(q) + r \geq \beta_m - M(1 - h).$$

类似地，任意两台 AMR 的潜在 footprint 冲突对需要满足时间错开约束。若

$$\|X_q(r) - X_{q'}(r')\| < 2\rho + \sigma,$$

则应有

$$departure(k, q) + r + \epsilon \leq departure(k', q') + r' + Mh$$

或

$$departure(k', q') + r' + \epsilon \leq departure(k, q) + r + M(1 - h).$$

完整时空 MILP 会产生大量二元互斥变量，因此工程实现采用“候选调度生成 + 轨迹检测 + 等待修复”的滚动时域启发式过程。

7.6 字典序目标规划

P4 的评价顺序采用目标规划思想，而不是单一加权和。硬约束层优先于所有经济性指标：

$$F_0 = N_{conflict} + N_{block} + N_{delay} + N_{missing} + N_{battery}.$$

其中 $N_{conflict}$ 为剩余轨迹冲突数， N_{block} 为封锁区违规数， N_{delay} 为 AMR 延误窗口违规数， $N_{missing}$ 为缺失路径数， $N_{battery}$ 为电量安全阈值违反数。只有 $F_0 = 0$ 的方案才具有运营意义。

在可行方案中，P4 进一步比较服务质量和运行成本。当前实现中的均衡目标可概括为

$$\min \text{lex} \left(F_0, \sum_i T_i, \sum_i w_i L_i, \sum_i L_i, C_{\max}, \sum_i H_i, F_3 \right),$$

其中

$$F_2 = 30 \sum_i w_i T_i + 20 \sum_i T_i + 5C_{\max},$$

$$F_3 = 2 \cdot \text{empty_cost} + 10(L_{\max} - L_{\min}) + \text{total_energy}.$$

报告表格中的 F_2 和 F_3 分别对应时间效率综合指标和运营成本综合指标。

此外，P4 还保留两类计划稳定性变量。若任务 i 的 AMR 分配相对 P3 基准发生变化，或开始/完成时间偏移超过容差，则记为扰动任务。开始时间偏移可写为

$$\Delta_i \geq S_i - S_i^0, \quad \Delta_i \geq S_i^0 - S_i.$$

稳定性目标不应高于硬约束和服务质量目标：允许少量扰动换取消除违规和降低总延期，但不允许为了减少扰动而保留不可执行方案。

7.7 求解策略与实现流程

考虑到完整时空 MILP 的变量规模随候选路径、任务数和时间离散粒度快速增长，P4 采用滚动时域启发式求解。整体流程如下：

1. 读取 P3 基准计划和轨迹，建立任务、AMR、路径和轨迹样本索引。

2. 按事件时刻排序处理动态事件，得到当前滚动窗口 t_e 。
3. 根据轨迹和任务时间识别事件影响范围，并划分 J_{fix} 与 J_{free} 。
4. 基于固定任务构造 AMR 当前状态 $\xi_k(t_e)$ 。
5. 对封锁和延误事件，取受影响任务及其同车后续尾段作为重排池；对新增任务，枚举其在未来任务序列中的插入位置。
6. 通过后悔值插入、局部顺序保留和新任务短名单修复构造候选序列。
7. 对候选序列重新选择 P1 mixed 路径并生成未来轨迹。
8. 调用 P3 的轨迹冲突检测函数；若存在冲突，则对未来任务增加等待，直到冲突数为 0 或达到迭代上限。
9. 合并固定轨迹和未来轨迹，输出新计划，并进入下一个事件。

其中，新任务插入是当前 P4 的关键。P4 采用“冲突感知短名单 + 有界修复后比较”：首先枚举新任务在每台 AMR 未来序列中的所有插入位置，生成临时轨迹并统计潜在冲突；随后保留全局前若干候选、每台 AMR 的代表候选和每台 AMR 的队首插入候选；最后对短名单做有限轮冲突修复，并按修复后的字典序目标选择方案。该策略把搜索集中在最有价值的候选位置上，同时保留初始冲突较多但修复后目标更优的方案。

该求解策略是面向动态仓储运行的可解释滚动优化启发式。它的优势在于：第一，事件到来后只重排未来任务，保持历史计划稳定；第二，所有路径仍来自 P1 mixed 候选库，接口清晰；第三，最终轨迹重新经过 P3 冲突检测和动态事件违规检查，能够给出可行性证据；第四，候选修复被限制在短名单和有限迭代内，使运行时间在本算例中保持在几十秒以内。

7.8 与 P2/P3 的区别

P4 与 P2/P3 的关系容易混淆，因此需要明确其职责边界。P2 解决的是静态任务分配与排序问题，核心决策是“任务由谁做、按什么顺序做、走哪条候选路径”。P3 不改变 P2 的任务归属，重点检查二维时空轨迹是否冲突，并通过等待、换路和局部换序修复执行层冲突。P4 则发生在动态事件到来之后：它不重做所有历史任务，而是把动态事件后的未来任务视为一个滚动窗口，在该窗口内重新安排受影响任务和新增任务。

因此，P4 的评价不能只看任务表是否更新，还必须同时看三类硬约束是否清零：轨迹冲突、封锁区违规和 AMR 延误违规。只有当这三类指标都为 0 时，P4 的延期、扰动任务数和 $C_{\max}$ 才有比较意义。这也是本文在实验分析中把可行性指标放在服务质量指标之前的原因。

8 P5：灵敏度分析

本部分围绕多 AMR 仓库调度系统的性能退化原因开展灵敏度分析。全文保持与前序调度流程一致，围绕 AMR 数量、任务负荷与瓶颈通道容量三个可解释参数，判断系统性能恶化究竟来自资源不足还是局部拥堵。

多 AMR 仓库调度的运行效果由资源配置与约束紧性共同决定。本部分保持与前序调度流程一致，围绕多 AMR 仓库调度系统的性能退化原因开展灵敏度分析。在本项目中，仓库拓扑、任务规则和动态事件机制给定后，调度系统的主要可控因素集中在 **AMR 数量、任务负荷和通道通行能力**。AMR 数量决定可用运输资源，任务负荷决定资源需求强度，通道容量决定局部路网约束。三者均可由工程措施直接改变，并且会同时影响任务分配、车辆排序、路径占用和动态重排结果。

在实际运营中，我们需要知道真正制约系统的资源瓶颈。若 **增加 AMR** 后完工时间改善明显，扩车具有直接价值；若任务量增加后等待和重排快速上升，说明系统已接近 **任务负荷边界**；若少数通道长期高占用并引发冲突，说明应优先关注 **局部通行能力**。灵敏度分析的意义就在于把这些现象拆成可比较的实验结果，支撑后续投入选择。

因此，我们以当前仓库布局、任务集合和动态事件为基准，依次进行 **基线校验、AMR 数量边际分析、任务负荷边界识别、热点通道容量检验**和 **工程干预对比**。最终结论服务于三个直接决策：是否继续扩充 AMR，是否控制单位周期内的任务释放量，是否优先改造瓶颈区域。

8.1 基线校验与占用热点识别

进行关键资源灵敏度分析之前，我们首先固定当前实例的**真实调度结果**。基线工况包含当前仓库布局、12 个静态任务、4 台 AMR，以及已注入的动态事件。系统依次完成**静态排程、冲突修复和动态重排**；同时将轨迹点投影到网格单元，统计**轨迹占用热点**。关键结果汇总见表 14 和表 15。

表 14: 基线工况关键结果

模块	任务 / AMR	$C_{\max}$
静态排程	12 / 4	40.840111
冲突修复	12 / 4	46.596970
动态重排	13 / 4	61.918082

表 15: 基线轨迹占用热点

热点	网格坐标	占用次数
1	(6, 4)	166
2	(5, 4)	152
3	(9, 1)	95
4	(9, 2)	84
5	(8, 2)	77

由表 14 可见，冲突修复使 $C_{\max}$ 从 40.840111 上升到 46.596970，增幅约 14.1%；动态重排后 $C_{\max}$ 进一步上升到 61.918082，较冲突修复结果增加约 32.9%。与此同时，总延期、剩余冲突和硬违规均为 0，说明**基线排程具有可行性**；变更任务 4 个、新

增任务 1 个，则说明**动态事件已经带来可观的计划扰动**。表 15 显示，(6,4) 与 (5,4) 两个相邻网格占用最高，分别达到 166 次和 152 次；(9,1)、(9,2) 和 (8,2) 也形成**连续的高占用区域**。

为解释完工时间上升背后的空间原因，我将轨迹点投影到**网格单元** Ω_{ab} ，并统计**占用次数与归一化占用密度**：

$$H_{ab} = \sum_{k=1}^N \mathbf{1}\{(x_k, y_k) \in \Omega_{ab}\}, \quad \rho_{ab} = \frac{H_{ab}}{\sum_{p,q} H_{pq}}. \quad (1)$$

对应空间分布见图 10。图 10 集中可视化展现了前文的**运筹策略与资源占有情况**。

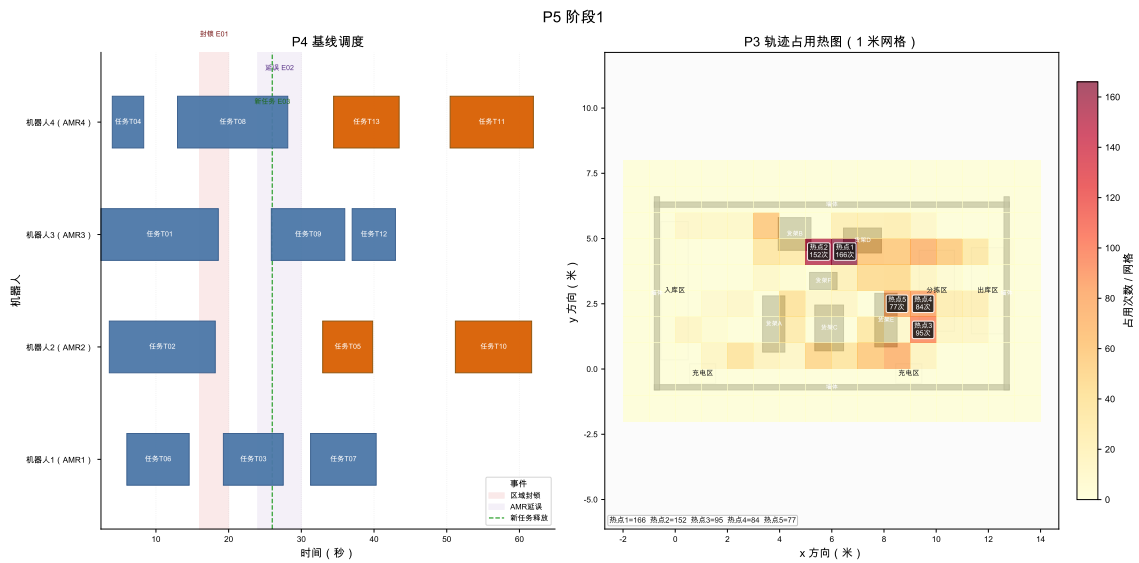


图 10: 基线工况下的重排甘特图与轨迹占用热图

综合表 14、表 15 和图 10，当前系统已经满足可行性校验，同时表现出局部拥堵与动态扰动。灵敏度分析因此围绕 **AMR 资源冗余、任务负荷边界和瓶颈通道容量** 展开。

8.2 AMR 数量的资源边际分析

AMR 数量是仓库调度系统中最直接的**资源变量**。车辆不足时，任务会在少数 AMR 上串行排队，**时间窗延期和最大完工时间同时上升**；车辆过多时，继续扩车只能带来**有限吞吐裕度**。因此，本部分固定仓库布局、12 个任务、时间窗、电量规则和 mixed 路径代价，只改变整数资源参数 m ，分析 m 的**边际响应**。

AMR 数量 m 为**离散整数资源**，局部灵敏度采用**枚举重优化后的前向有限差分**。对任一随 m 变化的性能指标 $G(m)$ ，定义

$$\Delta^+ G(m) = G(m) - G(m+1) \quad \text{一台 AMR 增量响应.} \quad (2)$$

我们以**运筹调配问题**中最关心的**最大完工时间**为核心性能指标，并定义

$$\begin{aligned}\Delta^+ C_{\max}(m) &= C_{\max}(m) - C_{\max}(m+1), \\ r_m &= \frac{\Delta^+ C_{\max}(m)}{C_{\max}(m)}, \\ \varepsilon_m &= \frac{\Delta^+ C_{\max}(m)/C_{\max}(m)}{1/m} = m r_m.\end{aligned}\tag{3}$$

$\Delta^+ C_{\max}(m)$ 表示完工时间下降量， r_m 表示相对下降比例， ε_m 表示**离散弹性近似**。

为判断上述改善是否真正消除**服务风险**，结合 AMR 调运的项目具体情况构造**服务状态指标**。设 $\mathcal{J}$ 为任务集合， $\mathcal{K}_m$ 为 AMR 集合， $|\mathcal{K}_m| = m$ ；参数 θ_0 表示固定的布局、路径代价、时间窗、服务时间、初始位置和电量规则。给定 m 后，调用主求解流程并记返回方案为 $\hat{\pi}_m$ 。评价器对 $\hat{\pi}_m$ 进行**时间和电量仿真**，得到任务完成时刻 $C_j(m)$ 、任务最晚完成时刻 ℓ_j 、路径缺失次数和电量违规次数。定义

$$\begin{aligned}D_j(m) &= \max\{0, C_j(m) - \ell_j\}, \\ C_{\max}(m) &= \max_{j \in \mathcal{J}} C_j(m).\end{aligned}\tag{4}$$

$D_j(m)$ 表示任务延期， $C_{\max}(m)$ 表示最大完工时间。据此定义服务可行性指标

$$\begin{aligned}S(m) &= \sum_{j \in \mathcal{J}} D_j(m), \\ L(m) &= \sum_{j \in \mathcal{J}} \mathbf{1}\{D_j(m) > 0\}, \\ M(m) &= N_{\text{trans}}^{\text{miss}}(m) + N_{\text{load}}^{\text{miss}}(m), \\ E(m) &= N_{\text{energy}}^{\text{viol}}(m), \\ Q(m) &= \mathbf{1}\{S(m) = 0, L(m) = 0, M(m) = 0, E(m) = 0\}.\end{aligned}\tag{5}$$

$S(m)$ 表示总延期， $L(m)$ 表示延期任务数， $M(m)$ 表示路径不可达次数， $E(m)$ 表示电量违规次数， $Q(m)$ 表示**服务可行性**。为辅助判断**资源是否紧张**，记 $A_{\text{act}}(m)$ 为空驶、载货行驶和服务时间之和，并定义

$$\rho(m) = \frac{A_{\text{act}}(m)}{m C_{\max}(m)}.\tag{6}$$

$\rho(m)$ 表示**平均资源利用率**。

图 11 同时展示**资源前沿**、**有限差分**、**服务风险**和**车辆负载**。4 台 AMR 是服务风险消除点；5 台以后 $C_{\max}$ 仍下降，但边际改善明显减弱。

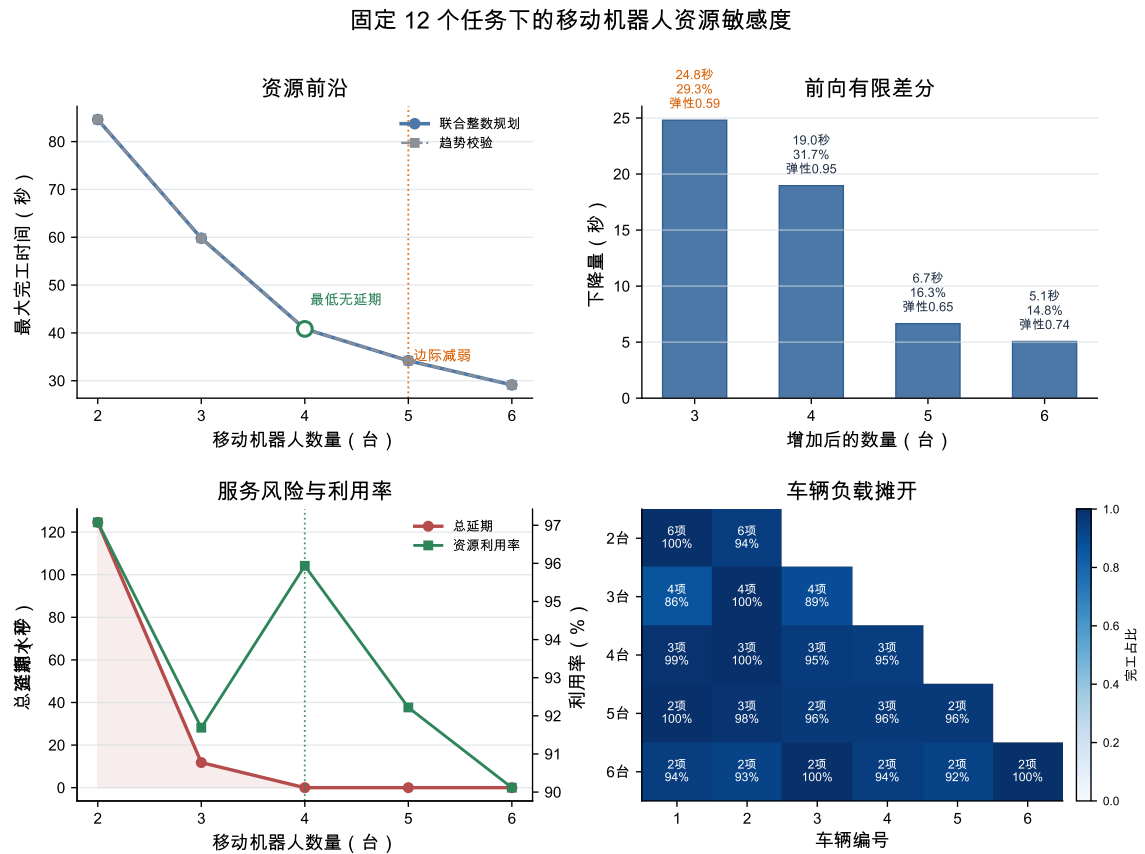


图 11: 固定任务集下 AMR 数量的资源前沿、前向有限差分、服务风险与车辆负载

表 16: AMR 资源前沿

m	$C_{\max}$	S	L	ρ	Q
2	84.606	124.593	4	97.08%	否
3	59.802	11.802	1	91.69%	否
4	40.840	0.000	0	95.94%	是
5	34.189	0.000	0	92.22%	是
6	29.130	0.000	0	90.12%	是

表 17: 增加一台 AMR 的前向有限差分

区间	$\Delta^+ C_{\max}$	r_m	ε_m	跨越
2-3	24.804	29.32%	0.586	否
3-4	18.962	31.71%	0.951	是
4-5	6.651	16.29%	0.651	否
5-6	5.059	14.80%	0.740	否

表 16 与表 17 表明，2 台和 3 台 AMR 均存在服务风险， $m = 4$ 时总延期、延期任务数、路径缺失和电量违规同时降为 0，首次达到可行服务状态。前向有限差分显示，3 到 4 台是关键敏感区间，虽然 $\Delta^+ C_{\max}$ 小于 2 到 3 台，但完成服务阈值跨越，且离散弹性达到 0.951。4 台以后继续扩车只降低 $C_{\max}$ ，不再改变服务可行性，因此 4 台 AMR 是当前算例的服务阈值，5 台可作为高裕度工况参考。

8.3 任务负荷与系统容量边界

在 AMR 数量固定后，进一步需要回答的是当前系统在一个业务时域内最多能稳定处理多少任务。这里不复用 P2 已有的任务密度灵敏度实验结果，而是在 P5 中单独构造嵌套任务池并逐点重优化；P2 仅作为单个工况的任务分配与排序求解器被调用。这

样可以保证任务负荷参数 n 的变化具有**严格可比性**，并且**容量边界来自同一套业务判定规则**。

本阶段固定仓库布局、mixed 路径库和 4 台 AMR。业务时域取

$$H = 55 \text{ s}, \quad (7)$$

其来源是基线静态任务批次的最大最晚完成时刻，表示当前任务波次的**服务承诺时域**。对每个随机种子 s ，P5 先生成最大任务池 $T_s(N)$ ，然后只取前缀形成不同规模的任务集合：

$$T_s(n) = \{j_1, \dots, j_n\}, \quad T_s(n) \subset T_s(n+1). \quad (8)$$

因此， n 增大时只是在同一个任务池上增加新任务，而不是更换另一批任务样本。对每个 (s, n) ，调用同一 P2 静态优化入口得到完工时间 $C_{\max, s}(n)$ 、总延期、路径缺失和电量违规。**容量判定采用保守的 all-seeds 规则**：

$$I_s(n) = \mathbf{1}\{C_{\max, s}(n) \leq H, \text{feasible}_s(n) = 1\}, \quad n_{\text{capacity}} = \max\{n : \prod_s I_s(n) = 1\}. \quad (9)$$

为刻画任务负荷的局部响应，采用**离散重优化后的前向有限差分**。令 $\tilde{C}_{\max}(n)$ 表示多种子中位数完工时间，并定义

$$\Delta_n C_{\max} = \tilde{C}_{\max}(n+1) - \tilde{C}_{\max}(n), \quad e_n = \frac{n}{\tilde{C}_{\max}(n)} \Delta_n C_{\max}. \quad (10)$$

$\Delta_n C_{\max}$ 表示增加一个任务带来的**边际完工时间增量**， e_n 是归一化后的**任务负荷弹性**。该指标来自**离散工况重优化结果**，不使用插值或平滑曲线替代实际求解点。

表 18: 任务负荷容量边界附近的离散扫描结果

n	$\min C_{\max}$	median $C_{\max}$	$\max C_{\max}$	$\Delta_n C_{\max}$	all-seeds
12	40.840	40.840	40.840	4.884	是
13	44.927	45.724	45.724	4.202	是
14	49.840	49.926	50.465	6.142	是
15	51.537	56.068	58.345	3.654	否
16	55.512	59.723	66.559	6.959	否

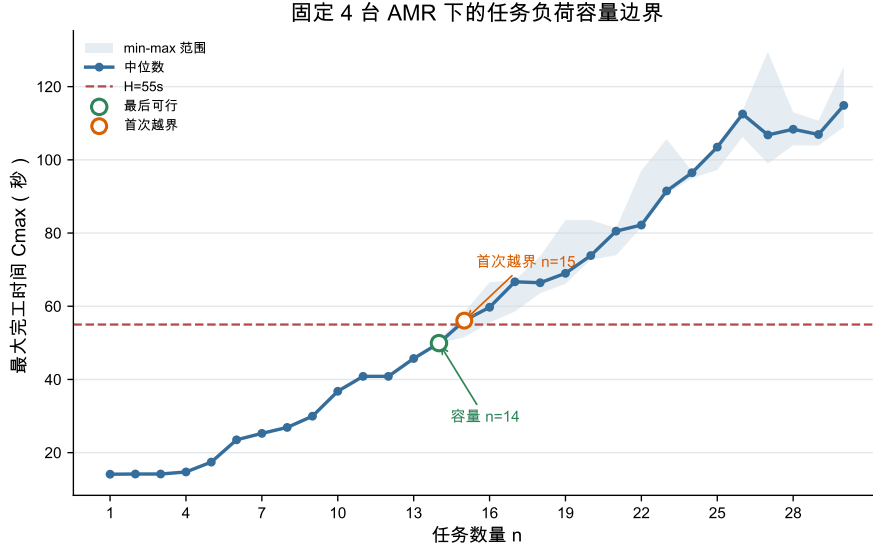


图 12: 固定 4 台 AMR 下的任务负荷容量边界

图 12 给出 $n \mapsto C_{\max}(n)$ 的容量边界曲线。蓝线为多种子中位数，浅色带表示 min-max 范围，红色虚线为统一业务时域 H 。由表 18 可见， $n = 14$ 时三个种子均满足 $C_{\max,s}(n) \leq 55$ 且无硬约束违规； $n = 15$ 时中位数已经达到 56.068222，且只有 1 个种子仍满足时域约束。因此，在固定 4 台 AMR 和 mixed 路径库下，本轮离散重优化得到的保守容量边界为

$$n_{\text{capacity}} = 14, \quad n_{\text{cross}} = 15. \quad (11)$$

需要注意的是， $\Delta_n C_{\max}$ 是每个负荷水平重新优化后的离散响应；个别高负荷点可能因任务组合和整数重优化结果变化出现非单调有限差分，因此不对曲线做单调化处理，也不使用插值曲线替代离散求解结果。

8.4 瓶颈空间簇与容量影子价值

任务负荷容量边界给出了系统最多能承受的任务规模，但仍需进一步定位限制调度性能的空间资源。

8.4.1 边界工况与离散影子价值

本节选择前文任务负荷与系统容量边界中接近容量边界且仍满足业务时域的工况，即 $n_{\text{capacity}} = 14$ 、方法为 M2 的种子 1 工况，并在该工况上重新展开 P3 时空冲突修复。我们把仓库局部通道视为有限容量资源，计算其容量单位放松后的边际价值。

设 C 为空间网格连通簇， $N_{C,t}$ 为时间桶 t 中占用簇 C 的 AMR 数。基线容量约束抽象为

$$N_{C,t} \leq q_C, \quad q_C = 1. \quad (12)$$

这里的 q_C 是局部能同时通过的 AMR 数量。执行 $q_C : 1 \rightarrow 2$ 可以解释为通道拓宽、设置会车区、调整局部交通规则或增加分拣口前缓冲区。由于当前 P2 的 MILP 没有显式

建立边容量约束，本部分对候选空间簇逐一放松容量并重跑 P3 冲突修复优化，得到**离散影子价值**

$$\pi_C^J = J^{\text{base}} - J^{+1}(C), \quad \pi_C^T = C_{\max}^{\text{base}} - C_{\max}^{+1}(C), \quad (13)$$

其中 P3 修复目标为

$$J = 100D_{\text{sum}} + 5C_{\max} + W + 2R + 20S + 20W_0. \quad (14)$$

D_{sum} 为总延期， W 为冲突修复插入等待， R 为换路次数， S 为同车换序次数， W_0 为初始等待。 π_C^J 和 π_C^T 分别表示簇容量增加 1 后目标函数和最大完工时间的真实改善量；**最终瓶颈排序以重优化后的有限差分为准，而不是以经过次数或热图颜色为准。**

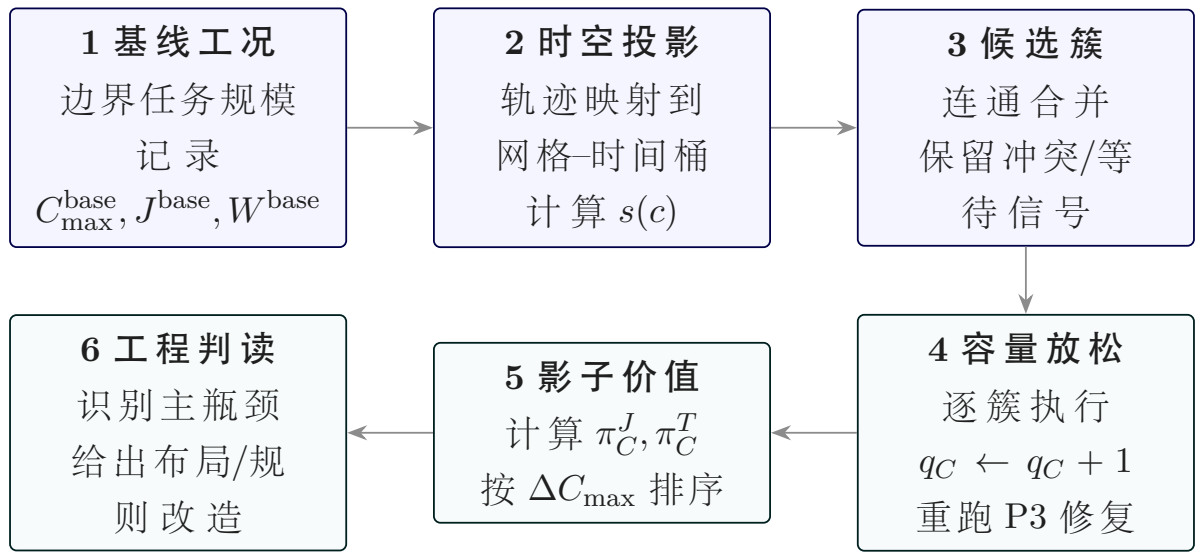


图 13: 瓶颈空间簇容量灵敏度分析流程

8.4.2 前置筛选

我们对候选簇进行空间统计。轨迹点按 $1\text{m} \times 1\text{m}$ 网格和 1s 时间桶投影，并对同一 AMR 在同一时间桶内的重复采样去重。对每个网格 c 计算占用、冲突修复事件、冲突等待和延期压力的标准化分量：

$$s(c) = 0.25z(\text{occ}_c) + 0.30z(\text{conf}_c) + 0.30z(\text{wait}_c) + 0.15z(\text{delay}_c). \quad (15)$$

相邻高分网格合并成连通簇后，只对含有冲突或等待信号的簇做容量放松重求。若一个区域只是经过次数高，但容量放松后 π_C^J 近似为 0，则只能称为高流量区，不能作为主瓶颈。

本工况中，轨迹、冲突、等待和延期信号共覆盖 56 个网格，其中 15 个网格的综合筛选分数为正。采用正分数网格的 70% 分位数作为阈值，得到 $s(c) \geq 1.731118$ 的高压力网格；再要求该网格必须含有冲突事件或等待信号，最终保留 5 个候选网格。由于这 5 个网格之间不相邻，连通合并后仍为 5 个单元簇，因此它们是由数据自动筛出的候选

集，而不是人工指定的固定数量。为避免漏检，又对路径库中其余 15 个未纳入网格执行同样的容量 +1 重优化，结果它们的 $\Delta C_{\max}$ 均为 0，说明当前主瓶颈没有被前面的筛选遗漏到更强的同类网格。

8.4.3 容量放松重优化

表 19: 候选瓶颈簇的容量放松有限差分

簇	位置	最近设施	筛选分数	$C_{\max}^{\text{base}}$	$C_{\max}^{+1}$	$\Delta C_{\max}$	ΔW
C03	(9, 2)	RACK_E / Z_SORT	1.776	83.870	69.142	14.728	49.0
C02	(6, 4)	RACK_D / Z_SORT	3.220	83.870	80.950	2.920	32.0
C04	(11, 4)	WALL_RIGHT / Z_OUT	2.416	83.870	83.870	0.000	0.0
C05	(12, 3)	WALL_RIGHT / Z_OUT	2.290	83.870	83.870	0.000	0.0
C01	(6, 2)	RACK_C / Z_SORT	2.319	83.870	83.870	0.000	0.0

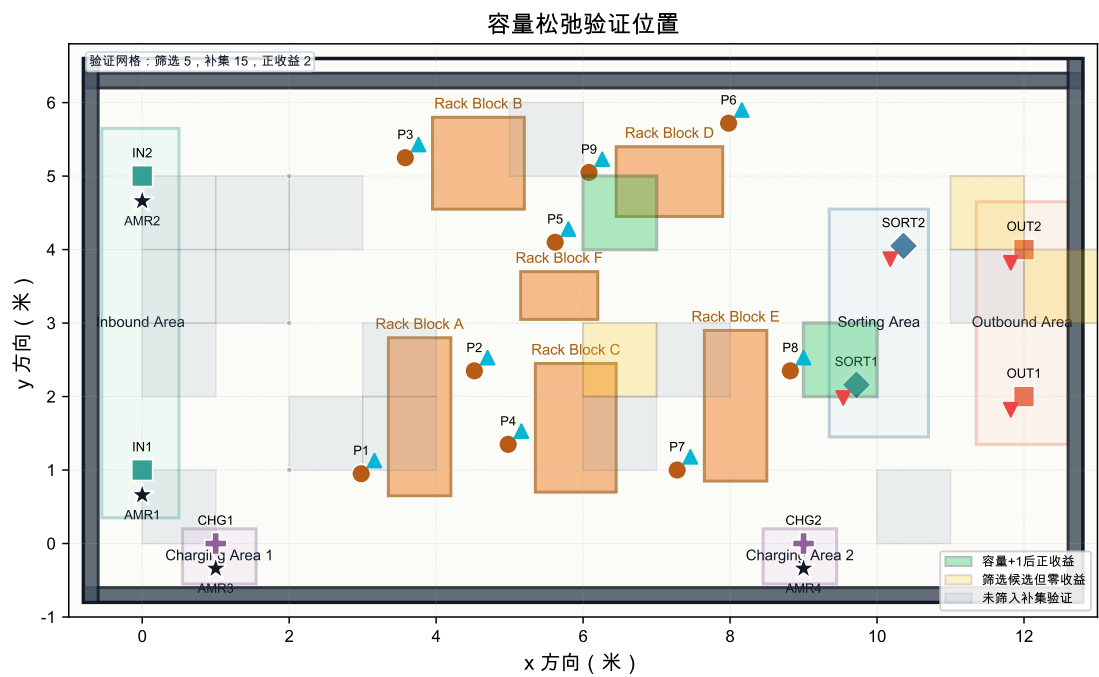


图 14: 瓶颈空间簇容量松弛验证位置

表 19 给出了筛选候选的容量放松结果，图 14 展示所有验证位置；路径库补集的 15 个未纳入网格在附加验证中全部得到 $\Delta C_{\max} = 0$ 。基线 P3 修复后 $C_{\max} = 83.869555$ ，冲突修复等待为 93.0 秒，目标函数为 17263.345675。对 5 个候选空间簇逐一执行容量 +1 并重跑 P3 后，C03 的边际改善最大： $C_{\max}$ 降至 69.141718，减少 14.727837 秒；冲突等待从 93.0 秒降至 44.0 秒，减少 49.0 秒；目标函数下降 10364.363885。C02 排名第二，仍有 2.919667 秒的 $\Delta C_{\max}$ 和 32.0 秒的等待改善。C04、C05 与 C01 的 $\Delta C_{\max}$ 均为 0，其中 C01 甚至出现目标函数轻微上升，说明其容量放松并未缓解当前主要拥堵结构。

8.4.4 瓶颈排序与工程含义

因此, C03 是当前边界工况下的主导空间瓶颈, 位置在 (9, 2), 对应 RACK_E 与 Z_SORT 的汇入带。这个结论来自容量松弛后的离散影子价值: 它同时在 $C_{\max}$ 、总等待和目标函数三个层面都给出最大的边际收益。工程上, 优先改造 C03 所在通道最有意义, 具体措施包括扩大 RACK_E 附近通道会车能力、在分拣区前设置短缓冲区、限制对向穿越, 或将进入 Z_SORT 的任务释放节奏做错峰控制。C02 可作为次级改造点继续评估, 而 C04、C05 与 C01 更适合保留监测而不是立即扩容。

8.5 Morris 与 Sobol 全局灵敏度筛选

本阶段将 AMR 数量、任务量和瓶颈簇容量作为同一组全局灵敏度分析因素: $Y = f(\theta) = C_{\max}(\theta)$, $\theta = (m, n, q_C)$, 其中 $m \in \{3, 4, 5, 6\}$, $n \in \{12, 13, 14, 15\}$, $q_C \in \{1, 2, 3\}$ 。瓶颈簇 q_C 自动取瓶颈空间簇与容量影子价值分析中排名第一的 C03。所有样本均运行真实前文运筹规划算法调度链路, 连续设计点 $x_i \in [0, 1]$ 仅用于抽样, 实际求解前按 $\mathcal{M}_i(x_i) = \ell_{i,r_i}, r_i = 1 + \min\{L_i - 1, \max\{0, \text{round}((L_i - 1)x_i)\}\}$ 映射到离散水平 $\mathcal{L}_i = (\ell_{i,1}, \dots, \ell_{i,L_i})$, 因此响应为 $Y = f(\mathcal{M}(x))$ 。

8.5.1 Morris 基本效应筛选

Morris 方法用有限差分基本效应筛选主导因素。设归一化空间划分为 p 个水平, 本实验 $p = 4$, 步长、第 r 条轨迹上因素 i 的基本效应、对 R 条轨迹汇总定义如下

$$\Delta = \frac{p}{2(p-1)} = \frac{2}{3}.$$

$$EE_i^{(r)} = \frac{f(\mathcal{M}(x^{(r)} + \Delta e_i)) - f(\mathcal{M}(x^{(r)}))}{\Delta}.$$

$$\mu_i^* = \frac{1}{R} \sum_{r=1}^R |EE_i^{(r)}|, \quad \sigma_i = \sqrt{\frac{1}{R-1} \sum_{r=1}^R (EE_i^{(r)} - \bar{EE}_i)^2}.$$

其中 μ_i^* 表示全局影响强度, σ_i 表示影响随背景工况变化的幅度; σ_i 较大通常意味着非线性效应或交互效应。

表 20: Morris 全局筛选结果

因素	μ^*	σ	μ^* 置信宽度
m	30.711	10.943	6.490
n	12.318	11.580	5.310
q_C	1.875	3.841	1.971

表 20 显示, m 的 $\mu^* = 30.711$ 秒, 明显高于 n 和 q_C ; n 的 $\sigma = 11.580$ 秒接近 m , 说明任务量效应强烈依赖车队规模和拥堵背景。从 Morris 筛选看, 增加 AMR 数量是当前工况下最直接的资源配置杠杆, 任务量是次级压力源, 单独提高 C03 局部容量的全局收益有限。

8.5.2 Sobol 方差分解验证

Sobol 方法从方差分解度量因素贡献。令 $X = (X_1, \dots, X_D)$ 且 $D = 3$, 若 $Y = f(\mathcal{M}(X))$ 平方可积, 则

$$f(X) = f_0 + \sum_i f_i(X_i) + \sum_{i < j} f_{ij}(X_i, X_j) + \dots, \quad V = \text{Var}(Y) = \sum_i V_i + \sum_{i < j} V_{ij} + \dots$$

一阶效应和总效应定义为

$$S_i = \frac{\text{Var}_{X_i}(\mathbb{E}_{X_{\sim i}}[Y | X_i])}{\text{Var}(Y)}, \quad S_{T_i} = 1 - \frac{\text{Var}_{X_{\sim i}}(\mathbb{E}_{X_i}[Y | X_{\sim i}])}{\text{Var}(Y)}.$$

S_i 表示因素 i 的独立方差贡献, S_{T_i} 表示包含全部高阶交互后的总贡献。本实验采用 Saltelli/Sobol 设计, $N = 64, D = 3$, 样本规模为 $N(2D + 2) = 512$ 。构造 $A, B, A_B^{(i)}$ 后, 典型估计量为

$$\hat{S}_i = \frac{\frac{1}{N} \sum_{k=1}^N y_B^{(k)} (y_{A_B^{(i)}}^{(k)} - y_A^{(k)})}{\hat{V}}, \quad \hat{S}_{T_i} = \frac{\frac{1}{2N} \sum_{k=1}^N (y_A^{(k)} - y_{A_B^{(i)}}^{(k)})^2}{\hat{V}}.$$

表 21: Sobol 方差分解结果

因素	S_1	S_1 置信宽度	S_T	S_T 置信宽度
m	0.752	0.343	0.880	0.362
n	0.276	0.215	0.350	0.132
q_C	-0.031	0.088	0.031	0.031

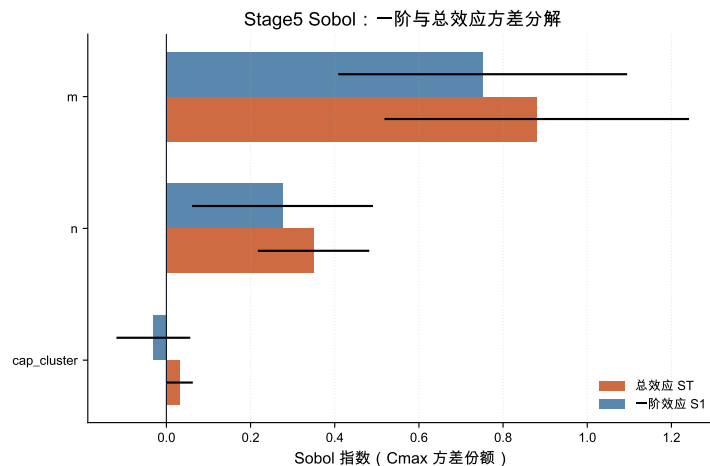


图 15: Sobol 一阶效应与总效应

表 21 和图 15 显示, m 的 $S_T = 0.880$, 显著高于 n 的 0.350 和 q_C 的 0.031。 q_C 的 S_1 轻微为负属于有限样本 Monte Carlo 估计误差, 不代表容量放松有负贡献; 结合其总效应接近 0, 可认为 C03 容量在本全局参数域内解释力很弱。**Sobol 验证了 Morris 排序, m 是主导因素, n 是次级因素, q_C 只适合做局部瓶颈修复。**

故灵敏度分析对仓库管理员的干预优先级作出指导, 即优先评估 AMR 数量扩容和任务负荷边界, 再对 C03 做针对性容量放松; 局部容量放松不能替代全局资源配置。

9 实验结果与分析

9.1 路径源对调度结果的影响

路径源对比显示, 不同 P1 算法会改变 P2 的调度质量。基础算例中, $m3@avg$ 的 C_{max} 为 40.373111, $m3@vg$ 为 40.547273, $m3@mixed$ 为 40.577778, $m3@davg$ 为 44.807677。DAVG 并非在静态调度中必然最优, 因为它引入动态风险代价后可能选择更保守路径; 其主要价值在动态风险或封锁场景中。

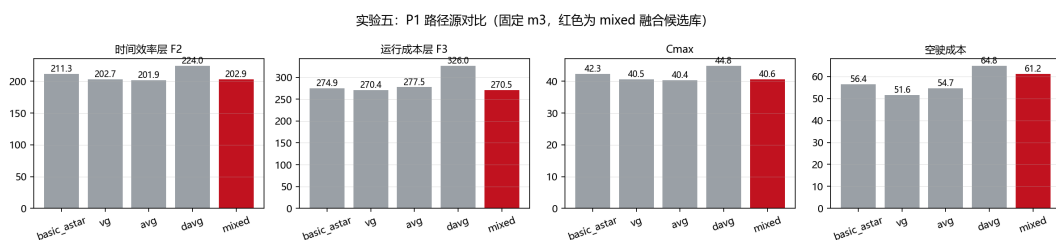


图 16: 不同 P1 路径源对 P2 结果的影响

图 16 反映 P1 和 P2 的联动关系: 路径源不是单纯的前处理细节, 而会直接影响上层调度目标。DAVG 在静态算例中因为动态风险代价更保守, 结果反而不一定最优; mixed 路径库的价值在于降低对单一路径算法的依赖, 使调度器可以在多种路径风格之间自动选择。

9.2 P2 方法对比

outputs/p2_experiments/p2_method_comparison.csv 记录了 P2 的方法对比和敏感性实验。基础算例中, $m0$ 、 $m1$ 、 $m2$ 、 $m3$ 都能得到无缺失路径、无电量违反、无延期的解, 但时间效率和运行成本不同。

方法	求解时间 (s)	$C_{\max}$	F2	F3	说明
m0	0.004	49.015333	245.076667	427.068565	贪心基线
m1	5.635	51.601313	258.006566	448.638042	两阶段精确/DP
m2	116.728	40.577778	202.888889	270.517511	联合 MILP 目标规划
m3	0.198	40.577778	202.888889	270.517511	后悔值插入 + 局部搜索

表 22: P2 基础算例方法对比

表 22 给出了 P2 基础算例的核心数值。可以看到，m2 与 m3 在基础算例上达到相同的 F2/F3 水平，但 m3 的运行时间显著更短。若以 m2 为精确建模标尺，则 m3 在该算例中以约 1/589 的时间得到同等质量解，说明本文并非只停留在模型公式层面，而是进一步给出了可扩展的工程求解路径。因此，m2 适合展示整数规划和目标规划建模，m3 适合在批量敏感性实验和较大规模场景中作为快速求解器。

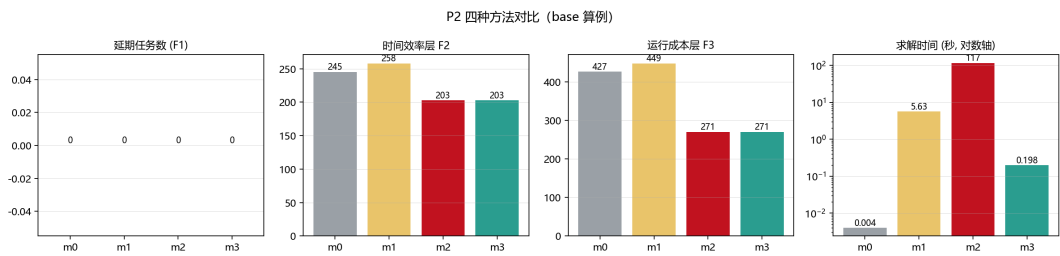


图 17: P2 方法对比图

图 17 将表 22 中的差异可视化。其关键信息有两点：第一，联合优化模型明显优于贪心和两阶段分解；第二，m3 在保持解质量的同时大幅降低求解时间。这说明精确模型和启发式方法不是相互替代，而是分别承担“建模基准”和“规模扩展”的角色。

9.3 P2 敏感性分析

AMR 数量敏感性实验表明，当 AMR 数量从 2 增加到 4 时，延期和 $C_{\max}$ 显著改善；继续增加 AMR 时，系统收益会逐渐受通道与任务结构限制。这一结论具有管理含义：仓库效率瓶颈并不总是“机器人不够”，当 AMR 数量达到一定水平后，继续增加车辆可能只会把瓶颈转移到通道、节点和任务释放节奏上。任务密度敏感性实验用于观察任务规模增加后延期、负载均衡和求解时间的变化，为选择 m2 或 m3 提供依据。

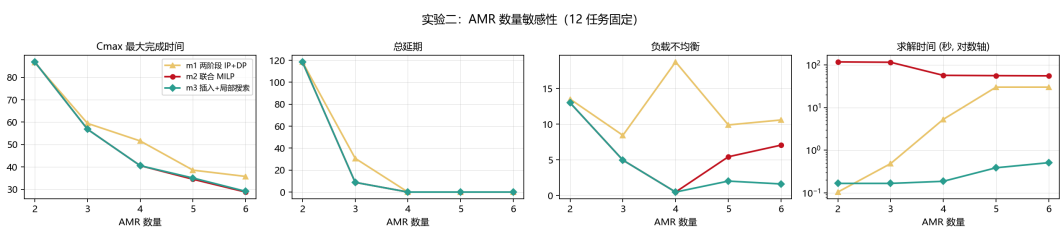


图 18: AMR 数量敏感性分析

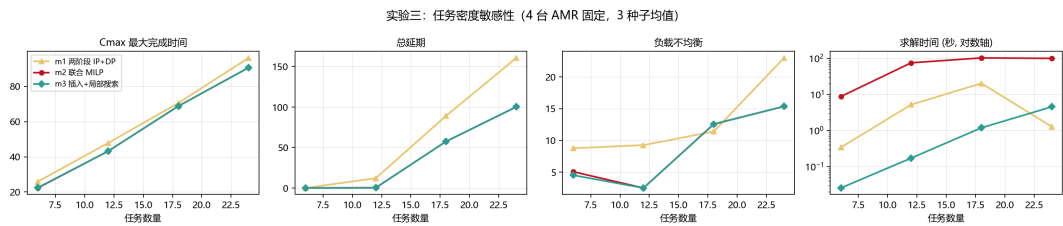


图 19: 任务密度敏感性分析

图 18 和图 19 分别从资源供给和任务压力两个方向检验模型稳定性。前者说明 AMR 数量增加存在边际收益递减，后者说明任务密度上升后延期和求解难度都会增加。两组实验共同支撑一个管理结论：仓库调度优化不能只看单次算例最优值，还要分析系统在不同资源和负载条件下的变化趋势。

9.4 P3 冲突修复结果

P2 静态计划在任务级别可行，缺失路径、电量阈值违反和任务延期均为 0，但直接展开为二维轨迹后仍存在 21 个初始冲突。P3 修复后，剩余冲突降为 0，总延期仍为 0，新增等待 16.0， $C_{\max}$ 从 P2 的 40.840111 增加到 46.596970。这说明任务级可行并不等价于轨迹级可执行；只有加入 P3 的二维时空检测与修复，调度结果才真正具备运行意义。

指标	数值
P2 原始计划展开后的冲突数	21
P3 修复后剩余冲突数	0
P3 任务总延期	0.0
P2 基准 $C_{\max}$	40.840111
P3 修复后 $C_{\max}$	46.596970
新增等待时间	16.0
轨迹采样记录数	1800
修复过程日志行数	15
实际换路次数	0
实际换序次数	0

表 23: P3 轨迹级修复核心指标

表 23 给出了 P3 的核心验证结果。P3 的主要代价是完工时间增加约 5.76 秒，而不是产生任务延期。由于所有任务仍满足最晚完成时间，P3 修复属于在服务水平不下降的前提下换取执行层安全性。若没有 P3，P2 的任务表虽然在时间窗和电量上可行，但实际执行时会在二维空间中产生碰撞风险；经过 P3 后，计划从“任务表可行”推进到“轨迹级可执行”。

从修复过程看，典型动作是中途等待让行：AMR2/T02、AMR3/T01 和 AMR4/T08

分别在载货路径上加入主要等待，使冲突数逐步下降并最终清零。等待压缩后，最终新增等待为 16.0 秒，其中 AMR2/T02、AMR3/T01 和 AMR4/T08 分别承担 4.0、4.0 和 8.0 秒。最终轨迹表共 1800 条采样记录，其中 `mid_path_wait_loaded` 有 163 条，说明 P3 不是简单整体延后任务，而是在载货路径靠近冲突点前插入停车让行片段。

该结果也揭示了 P3 的边界：当前算例主要验证了中途等待修复对轨迹冲突的消解能力，换路和同车局部换序虽然作为候选机制保留，但本次最终解中没有实际触发。若换成候选路径差异更明显、瓶颈通道更拥堵的地图，或在目标函数中提高等待与 $C_{\max}$ 权重、降低换路或换序惩罚，后两类动作才更可能体现收益。因此，本文将换路和换序表述为扩展能力，而不是当前实验的主要数值贡献。

从工程角度看，P3 的输出也为 P4 提供了稳定基准。P4 直接读取 P3 的 `schedule_result.csv` 和 `trajectory_schedule.csv` 作为输入。前者保留 P2 表头，便于继续读取任务级字段；后者记录最终二维轨迹，能够支持动态区域封锁、AMR 延误窗口和新增任务插入后的二次冲突判断。

综合来看，P3 的优势在于把抽象任务计划落到了二维轨迹层，并用可解释的局部动作消除了冲突；不足在于当前算法是启发式局部修复，不保证全局最优，且运行性能仍有优化空间。后续可以通过时间桶索引、局部重检和候选动作缓存降低计算成本，同时构造更复杂算例检验换路和换序机制的实际贡献。

9.5 P4 动态滚动重排结果

本节基于 `data/processed/p4` 的最新输出，对 P4 动态重排结果进行验证和解释。实验读取 12 个 P3 基准任务和 3 个动态事件，其中 2 个事件直接影响计划。P4 采用“可行性优先、服务质量其次”的评价顺序：轨迹冲突、封锁区违规、AMR 延误违规、缺失路径和电量阈值违反必须全部为 0；在此基础上，再比较延期、扰动任务数、 $C_{\max}$ 、 F_2 和 F_3 。最终滚动时域重排结果如下：

指标	数值
动态事件数	3
受影响事件数	2
新增动态任务数	1
变更或插入任务数	4
P3 基准 $C_{\max}$	46.596970
P4 新 $C_{\max}$	61.918082
P4 总延期 total_delay	23.405971
相 对 基 准 正 向 完 工 偏 移	43.996446
TotalAddedDelay	
剩余轨迹冲突	0
封锁区违规	0
AMR 延误违规	0
缺失路径数	0
电量阈值违反数	0
F2	2182.068048
F3	507.179375

表 24: P4 滚动重排主结果

表 24 展示动态事件后的最终可行性和服务质量。这里需要区分两个指标： total_delay 是按任务时间窗计算的延期总量，数值为 23.405971； TotalAddedDelay 是相对 P3/新任务基线的正向完工时间偏移总和，数值为 43.996446。二者含义不同，不能混写。

9.6 动态事件影响范围

P4 首先对三个动态事件逐一做影响分析，结果见表 25。影响分析把事件分为“触发重排”和“记录但不扰动”两类。E01 定义了封锁区域和封锁时间窗，但最终轨迹 footprint 没有进入该矩形区域，因此影响任务数为 0；E02 是 AMR2 在 [24, 30] 内延误，影响 AMR2 的 T05 和 T10；E03 是新增任务 T13，最终插入 AMR4。

事件	类型	目标	受影响任务	影响解释
E01	area_block	BLOCK_01	无	封锁窗口内没有轨迹 footprint 与封锁矩形相交
E02	amr_delay	AMR2	T05, T10	AMR2 在 [24, 30] 延误，T05 与该窗口重叠，T10 属于同车后续尾段
E03	new_task	T13	T13	新任务进入滚动窗口，最终插入 AMR4

表 25: P4 动态事件影响范围

这一结果表明，P4 采用事件影响驱动的重排触发机制。封锁事件只在轨迹 footprint

与封锁矩形发生时空交叠时触发重排；AMR 延误事件释放与延误窗口相交的任务及同车后续尾段；新增任务事件则进入未来任务池并参与插入位置搜索。这样的处理方式把动态事件转化为明确的重优化对象，使滚动窗口内的调整范围与实际受影响区域保持一致。

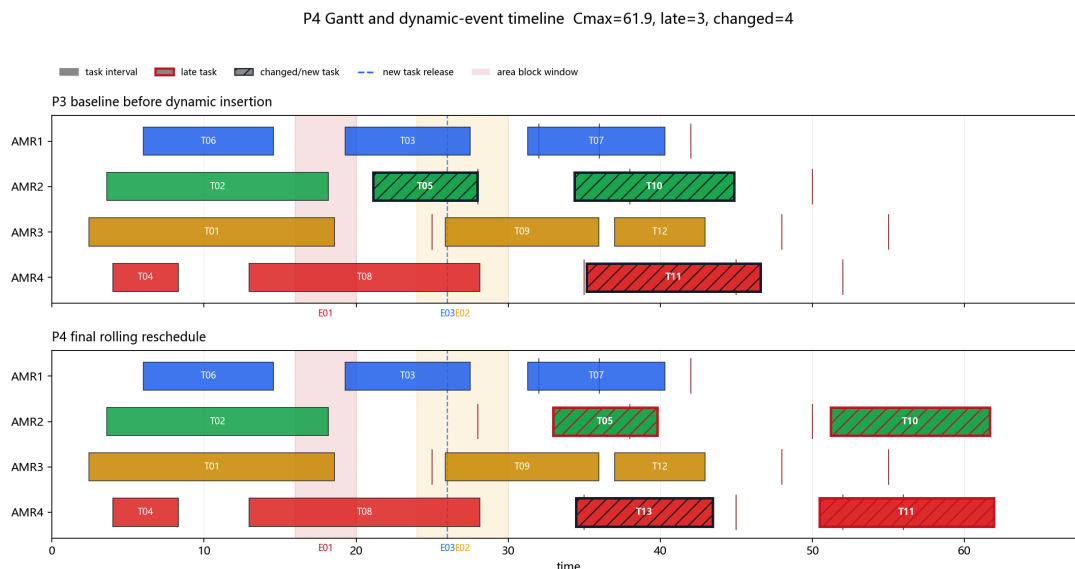


图 20: P4 动态事件与重排甘特图

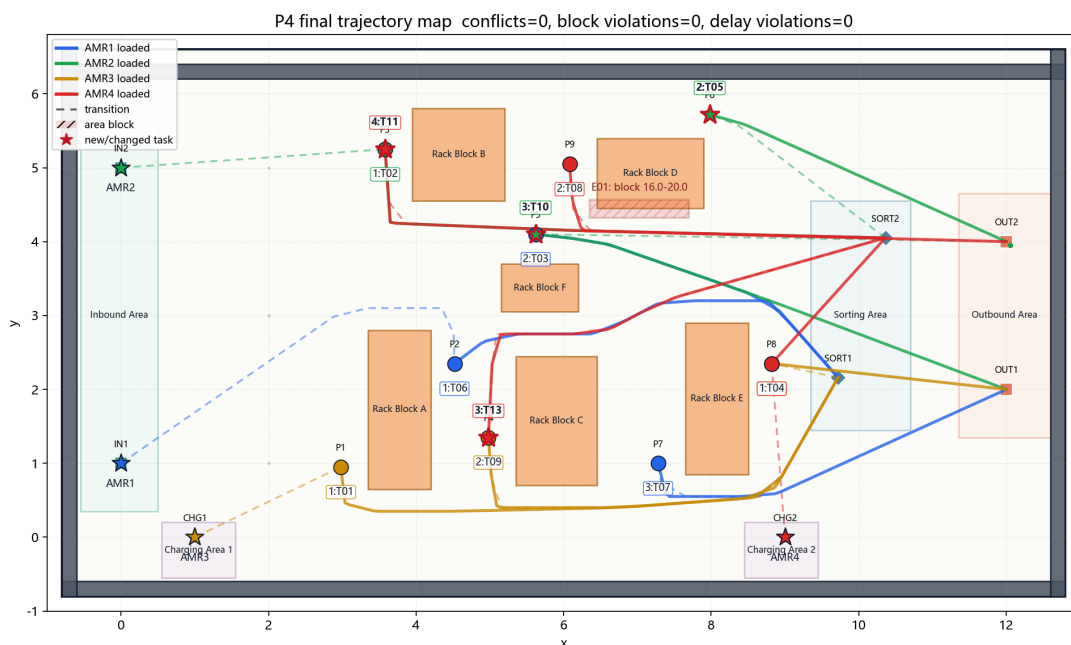


图 21: P4 重排后轨迹地图

图 20 从时间轴上展示动态事件与任务重排之间的关系，图 21 从仓库平面上展示重排后的轨迹空间分布，图 22 进一步把 GIF 动图中的四个关键时刻整理为静态快照。三张图共同展示 P4 对时间表、空间轨迹和动态过程的同步更新。



图 22: P4 动态滚动重排过程快照

该实验的关键变化是 AMR2 的 T05 与延误事件 E02 时间窗相交。P4 将 AMR2 的重叠任务及其后续尾段从固定集合中释放，并从延误结束后重新安排，最终得到 `delayed_amr_violation_count=0`。新增任务 T13 被插入 AMR4，并前置于 T11。该安排用 T11 的局部后移换取 AMR2 尾段负担下降，使滚动重排方案在硬约束可行的同时降低延期和完工时间。

9.7 重排任务明细

表 26 给出最终发生变化或新增的任务。P4 共改变或插入 4 个任务：T05、T10、T13 和 T11。其中 T05 和 T10 仍由 AMR2 执行，变化原因来自 E02；T13 是新增任务，被插入 AMR4；T11 原本也是 AMR4 任务，但由于 T13 被前置，T11 被推迟执行。

任务	原 AMR	新 AMR	原开始	新开始	原完成	新完成	延后量	原因
T05	AMR2	AMR2	21.101333	32.939000	27.972444	39.810111	11.837667	AMR2 延误后重排
T10	AMR2	AMR2	34.353444	51.191111	44.840111	61.677778	16.837667	AMR2 尾段重排, 并加入 5s 冲突等待
T13	—	AMR4	—	34.441920	—	43.453031	—	新增任务释放后插入 AMR4
T11	AMR4	AMR4	35.146465	50.467577	46.596970	61.918082	15.321112	T13 前置后被推迟

表 26: P4 重排任务明细

从表 26 可以看出, P4 同时完成了两类动态调整。E02 改变 AMR2 的可用状态, 使 T05 和 T10 从延误窗口之后重新排布; E03 引入新任务 T13, 并通过候选插入搜索选择

AMR4。当前解把 T13 放到 AMR4，并让 T11 后移，相当于用一个额外扰动任务换取更低的总延期和更短的系统完工时间。该选择符合 P4 的目标层次：先保持硬约束为 0，再在可行域内降低延期和 $C_{\max}$ 。

9.8 可行性验证

P4 最终方案的硬约束验证结果集中体现在表 27。表中的 0 来自最终 `trajectory_schedule.csv` 的轨迹级复核，覆盖 footprint 冲突、封锁区进入、AMR 延误窗口重叠、路径缺失和电量阈值违反。

指标	数值	含义
剩余轨迹冲突数	0	最终轨迹中无 footprint 或节点容量冲突
封锁区违规数	0	封锁区时间窗内无轨迹进入封锁矩形
AMR 延误违规数	0	AMR2 延误窗口内无任务执行区间重叠
缺失载货路径数	0	所有载货路径均能在 P1 mixed 路径库中找到
缺失空驶衔接路径数	0	所有空驶衔接路径均能在 P1 mixed 路径库中找到
电量阈值违反数	0	所有 AMR 完成任务后电量不低于安全阈值
全部任务已排程	1	静态任务和动态新增任务均已排入最终计划

表 27: P4 最终方案硬约束验证

表 27 给出最终动态可行性证据。AMR 延误违规数为 0，说明 E02 已被轨迹层完全消化：T05 原计划从 21.101333 执行到 27.972444，与 AMR2 的 [24, 30] 延误窗口重叠；P4 将其新开始时间推迟到 32.939000，从而避开延误窗口。T10 作为 AMR2 后续任务，也被相应推迟到 51.191111 开始。最终 AMR2 在不可用窗口内没有任务执行区间。

图 23 将动态事件影响、硬约束违反数和目标函数指标集中展示，图 24 则给出重排策略的对照输出。二者共同强调 P4 的评价顺序：先看轨迹冲突、封锁违规和 AMR 延误违规是否清零，再比较延期、扰动任务数和完工时间。这一顺序与实际仓储系统一致，因为不可执行的低成本方案没有运营意义。

9.9 目标函数指标解释

P4 的主要目标指标见表 28。`priority_late_count` 和 `late_count` 描述延期任务数量层面的服务质量；`priority_delay` 和 `total_delay` 描述延期幅度； $C_{\max}$ 表示系统

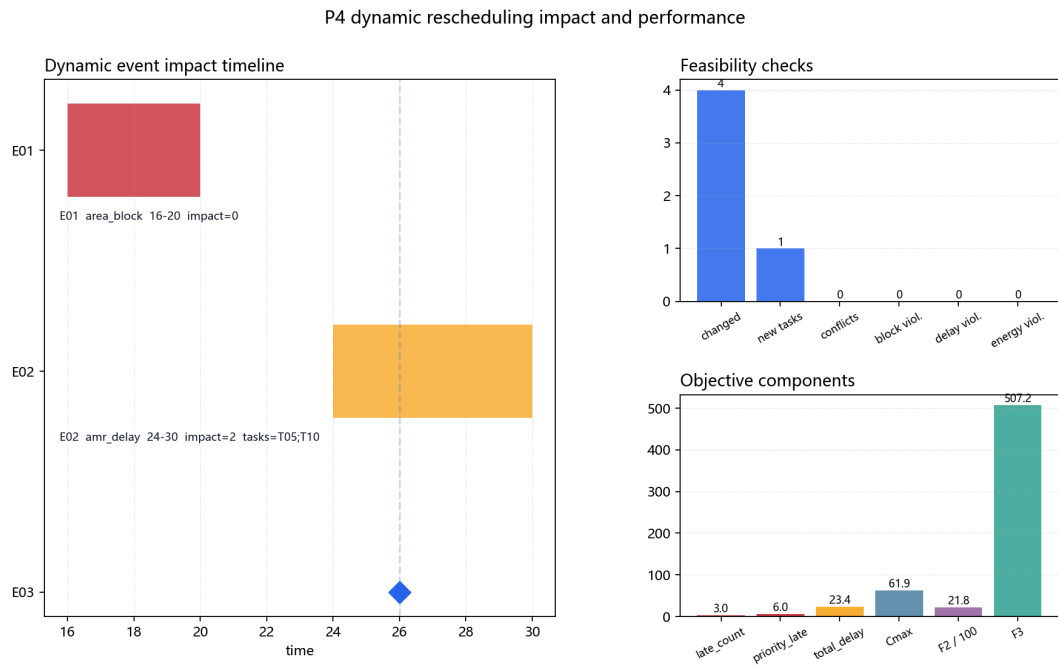


图 23: P4 事件影响与可行性指标

P4 dynamic rescheduling method comparison

方法	新增延期	总延期	Cmax	F2	冲突消解成功率
滚动时域重排	23.41	23.41	61.92	2182	100%

Feasibility is the first gate; among feasible methods, lower total delay, Cmax and F2 indicate better rolling performance.

图 24: P4 方法对比输出

最终完工时间； F_2 与 F_3 分别综合时间效率和运行成本。

指标	数值	解释
priority_late_count	6.0	延期任务按优先级加权后的数量
late_count	3	出现正延期的任务数量
priority_delay	46.811941	延期量按任务优先级加权后的总和
total_delay	23.405971	按任务软时间窗计算的总延期
$C_{\max}$	61.918082	P4 最终系统完工时间
F_2	2182.068048	$30 \sum_i w_i T_i + 20 \sum_i T_i + 5C_{\max}$
F_3	507.179375	空驶成本、负载不均衡和总能耗综合指标

表 28: P4 目标函数指标

动态事件改变了 P3 基准计划的可用时间和任务集合，特别是 AMR2 的不可用窗口压缩了其后续任务的可用区间。P4 先恢复硬可行性，再在可行域内优化服务指标；当前方案通过把 T13 插入 AMR4 来降低 AMR2 尾段负担。最终方案包含 3 个延期任务，但轨迹冲突、封锁违规和 AMR 延误违规全部为 0。

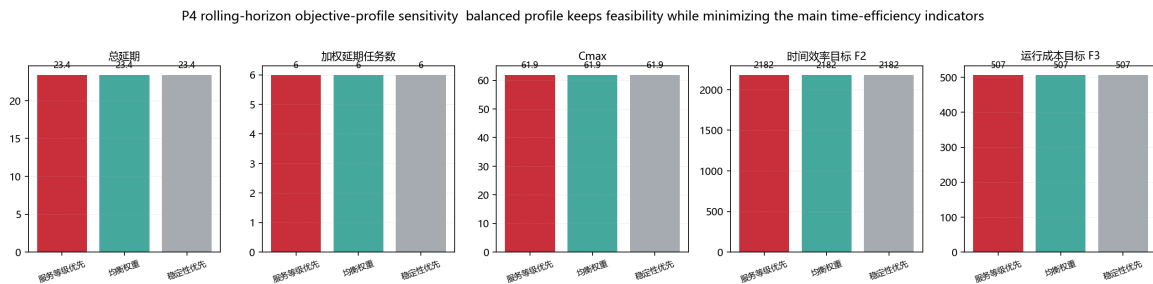


图 25: P4 目标权重配置敏感性结果

图 25 比较服务等级优先、均衡权重和稳定性优先三类目标配置。在当前算例中，三类配置均选择同一条可行重排方案，说明本算例的主导因素来自硬约束恢复和新增任务插入结构；局部权重变化没有改变最终候选选择。该结果支持本文采用均衡权重作为主报告方案。

9.10 与中间候选方案的比较

在调试和优化 P4 时，曾出现两个具有代表性的 AMR2 插入候选。第一个候选把 T13 插入 AMR2 并排在 T05 之前，虽然延期任务数较少，但会让 T05 和 T10 大幅后移，总延期为 41.794333， $C_{\max} = 73.331$ ， $F_2 = 3710.201667$ 。第二个候选把 T13 插在 AMR2 的 T05 与 T10 之间，总延期降为 27.105444，但 $C_{\max}$ 仍为 73.241，AMR2 尾段仍然较重。最终方案把 T13 插入 AMR4 并前置置于 T11，总延期进一步降到 23.405971， $C_{\max}$ 降到 61.918082。

方案	扰动任务数	总延期	$C_{\max}$	F_2
AMR2 前插 T13 候选	3	41.794333	73.331000	3710.201667
AMR2 中插 T13 候选	3	27.105444	73.241000	2596.270556
当前 AMR4 前插 T13 方案	4	23.405971	61.918082	2182.068048

表 29: P4 新任务插入候选效果比较

表 29 表明，当前方案以 1 个额外扰动任务换取了更低的总延期、更短的 $C_{\max}$ 和更优的 F_2 。相对 AMR2 中插候选，当前方案总延期降低

$$\frac{27.105444 - 23.405971}{27.105444} \approx 13.65\%,$$

$C_{\max}$ 降低

$$\frac{73.241000 - 61.918082}{73.241000} \approx 15.46\%.$$

相对最初 AMR2 前插候选，当前方案总延期降低约 44.00%， F_2 降低约 41.19%。这体现了 P4 短名单修复机制的作用：候选方案先经过轨迹检测和有限等待修复，再按字典序目标比较，因此能够保留初始冲突较多但修复后目标更优的插入位置。

9.11 图形结果解释

图 20 的主要作用是说明时间维度上的重排逻辑。AMR2 的 T05 被推迟到延误窗口之后，T10 作为同车后续任务继续后移；AMR4 在完成 T08 后插入 T13，再执行 T11。这个甘特图可以直接看出 P4 的滚动时域特点：事件发生前的已执行任务保持不变，事件后的未来尾段被局部调整。

图 21 则说明空间维度上的可行性。新增任务 T13 从 P4 到 SORT2，插入 AMR4 后没有造成封锁区违规，也没有留下轨迹冲突。轨迹图与甘特图配合使用，可以避免只从时间表判断动态重排是否成功。对多 AMR 仓储系统而言，时间上不重叠并不必然代表空间上安全，空间上绕开也不必然代表时间窗可接受；P4 必须同时满足两者。

图 23 将事件影响、硬约束违反和目标指标放在同一张图里，形成最终方案的可行性证据链。图 24 汇总滚动时域主结果，图 25 展示目标权重配置下的指标稳定性。三类图分别对应“事件影响”“方法结果”和“目标敏感性”，使 P4 的实验结论从时间、空间、目标三个层面得到支撑。

9.12 适用范围与扩展实验

P4 的实现定位为动态仓储运行中的在线滚动重排启发式。它把完整时空 MILP 中的任务指派、路径选择、等待和轨迹互斥约束转化为滚动窗口内的候选插入、短名单修复和等待冲突修复流程。该实现保留了混合整数规划的决策结构和目标层级，同时把求解范围限制在受影响任务池内，适合事件到达后快速恢复可执行计划。

当前解的数学含义是“给定候选集合与有界修复策略下的高质量动态可行解”。它的优势在于指标清晰、可复现、运行时间可接受，并且最终轨迹通过硬约束检查。在此基础上，可以进一步加入小规模精确模型作为离线上界对照：例如只对受影响 AMR 和相邻时间窗建立局部 MILP，或者把候选插入、等待动作离散化后，用 0-1 规划选择动作组合。

离线横向对比可以把“仅等待”“全局重排”“滚动时域重排”作为三类基线分别运行，并统一比较新增延期、扰动任务数、冲突消解成功率和运行时间。这样可以进一步量化滚动时域方法的收益来源：它在保持历史计划稳定的同时，对受影响未来尾段进行重优化，从而兼顾执行稳定性和动态服务质量。

总体而言，P4 已经完成动态仓储场景下的保底要求和进阶要求：能够识别封锁、延误和新增任务事件；能够释放受影响任务和未来尾段；能够将新增任务插入合适 AMR；能够输出结构化重排表、可行性指标和动态可视化；最终方案在轨迹冲突、封锁区违规和 AMR 延误违规上均为 0，并通过局部搜索将总延期控制在 23.405971。

10 结论

本项目最终形成了一套动态仓储多 AMR 调度优化系统。P1 负责二维候选路径生成，P2 将任务分配和任务排序建模为带路径选择、电量检查和目标规划优先级的调度问题，P3 从轨迹层面验证并修复多机器人冲突，P4 在动态事件下执行滚动时域重排，P5 对 AMR 数量、任务负荷和瓶颈通道容量开展灵敏度分析。

实验结果说明，P2 的任务级计划虽然已经满足路径、电量和时间窗要求，但展开到二维时空后仍会产生 21 个轨迹冲突；经过 P3 的等待修复后，冲突数降为 0。动态事件到达后，P4 能在不重排全部历史任务的情况下处理封锁、AMR 延误和新增任务，并得到剩余轨迹冲突、封锁区违规、AMR 延误违规均为 0 的滚动重排方案。

整体来看，项目将路径库、任务分配、时空冲突修复、动态扰动处理和灵敏度分析串联为一个统一接口体系。报告中的模型、算法和实验结果共同说明：任务级排程必须经过轨迹级校验，动态扰动需要在保持可执行性的前提下进行局部重排，P5 灵敏度分析进一步把这些结果转化为面向仓库运营的管理建议。

参考文献

- [1] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [4] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.
- [5] Tomas Lozano-Perez and Michael A. Wesley. An algorithm for planning collision-free paths among polyhedral obstacles. *Communications of the ACM*, 22(10):560–570, 1979.

A 成员分工

本附录汇总本项目五名成员在模型设计、程序实现、实验分析和报告撰写中的主要分工。各成员围绕 P0–P5 工作包协同完成，从二维场景建模、候选路径生成、任务分配排序、轨迹冲突修复、动态滚动重排到灵敏度分析，形成完整的多 AMR 调度优化链路。

成员	工作包	分工主题	主要贡献
李彦霖	P0/P1	场景建模与候选路径生成，PPT 制作	负责仓储二维场景、节点和障碍物数据整理，完成 A*、VG、AVG、DAVG 及 mixed 候选路径库构建，输出路径成本表和轨迹样本。
朱熠正	P2	任务分配与排序优化，报告框架搭建	负责多 AMR 任务指派、车内任务排序、路径选择和时间窗约束建模，比较贪心、两阶段整数规划、联合 MILP 与启发式局部搜索方法。
江亚楠	P3	时空冲突检测与修复	负责将任务级计划展开为二维时空轨迹，检测 AMR footprint 冲突和节点容量冲突，并通过等待、换路和换序策略完成冲突修复。
倪振昊	P4	动态事件与滚动重排	负责通道封锁、AMR 延误和新增任务等动态事件建模，在冻结已执行任务的基础上重排未来任务池，并完成动态重排结果可视化。
张铭浥	P5	灵敏度分析与管理解释	负责基线校验、AMR 数量边际分析、任务容量边界、瓶颈通道容量和 Morris/Sobol 全局灵敏度分析，形成运营管理建议。

表 30: 小组成员分工汇总